Architettura, Modello e Analisi dei Sistemi Cyber-Fisici

Dispensa del Corso – Marco Pietri

> Documento creato come integrazione alle slide del corso. Contiene i concetti fondamentali, arricchiti con spiegazioni aggiuntive e organizzati in modo da facilitare lo studio.

CAPITOLO 1 – Sistemi Distribuiti: Concetti Fondamentali

1.1 Cos'è un Sistema Distribuito?

Un **sistema distribuito** è un insieme di computer indipendenti che appare agli utenti come un singolo sistema coerente. L'obiettivo principale è nascondere la complessità interna e offrire un'esperienza d'uso uniforme, come se si stesse usando un solo computer.

Caratteristiche chiave

- **Assenza di un clock globale**: ogni macchina ha il proprio orologio interno, che tende a desincronizzarsi nel tempo a causa del *drift* dell'oscillatore. Questo rende la sincronizzazione uno dei problemi centrali dei sistemi distribuiti. Ad esempio, se l'orologio di un computer è impostato male, potrebbe non riuscire nemmeno a connettersi a certi servizi online.

- **Gestione dei guasti parziali**: un sistema distribuito deve essere progettato in modo che un guasto in una parte non comprometta il funzionamento dell'intero sistema. I guasti (o *fault*) non sono eventi eccezionali, ma vanno considerati come parte normale del comportamento del sistema.

- **Overlay Network**: le reti overlay sono reti logiche costruite sopra la rete fisica (Internet). Ne esistono di due tipi:
- **Strutturate**: i nodi sono organizzati secondo una topologia ben definita (es. DHT – Distributed Hash Table).
- **Non strutturate**: i nodi si connettono in modo casuale o semi-casuale (es. reti BitTorrent).

1.2 Trasparenza nei Sistemi Distribuiti

La **trasparenza** è la capacità del sistema di nascondere la propria natura distribuita agli utenti e agli sviluppatori. Un esempio concreto in C è la funzione `inet_ntoa()`, che garantisce che tutti gli utenti usino lo stesso protocollo di rete in modo trasparente.

Le principali forme di trasparenza sono:

Tipo di Trasparenza	Descrizione
Accesso	L'accesso alle risorse avviene allo stesso modo indipendentemente da dove si trovano
Posizione	L'utente non sa dove si trova fisicamente la risorsa
Migrazione	Le risorse possono spostarsi senza che l'utente se ne accorga
Rilocazione	Una risorsa può essere spostata mentre è in uso
Replica	Più copie della stessa risorsa esistono senza che l'utente ne sia consapevole
Concorrenza	Più utenti accedono alle stesse risorse senza interferirsi
Guasto	Il sistema continua a funzionare anche in presenza di errori

---

1.3 Apertura e Scalabilità

Apertura

Un sistema distribuito **aperto** è un sistema in cui tutti i componenti usano interfacce ben definite e standardizzate, in modo da poter comunicare con sistemi sviluppati in modo indipendente. Il requisito fondamentale è l'**interoperabilità**: diversi sistemi devono potersi scambiare dati e funzionalità senza conoscersi nei dettagli implementativi.

Scalabilità

La scalabilità è la capacità del sistema di gestire un numero crescente di utenti, dati o macchine senza degradare le prestazioni. Nelle soluzioni moderne la maggior parte dei problemi di scalabilità sono stati risolti, ma rimangono alcune sfide:

- **Scalabilità geografica**: la comunicazione tra paesi distanti è meno affidabile e più lenta.
- **Scalabilità amministrativa**: leggi e regolamenti diversi nei vari paesi possono creare ostacoli.

1.4 Tipologie di Sistemi Distribuiti

1.4.1 Parallel Computing (Calcolo Parallelo)

Si basa sull'utilizzo di più CPU che lavorano contemporaneamente sullo stesso problema. L'obiettivo è ottenere alte prestazioni sfruttando il parallelismo.

Problema principale: la memoria condivisa non scala bene. Quando troppi processi accedono contemporaneamente alla stessa area di memoria, si crea un *bottleneck*. La MMU (Memory Management Unit) viene usata per assegnare automaticamente lo spazio di memoria corretto a ciascun processo.

1.4.2 Cluster Computing

È simile al calcolo parallelo, ma con un **nodo master** che coordina il lavoro degli altri nodi. Il master distribuisce i task ai worker e raccoglie i risultati. Tutti i nodi sono tipicamente omogenei (stesso hardware e OS).

1.4.3 Grid Computing

Una **generalizzazione** del cluster e del calcolo parallelo. Le principali differenze rispetto al cluster:

- I nodi sono **eterogenei** (hardware e software diversi).
- È possibile usare solo **una parte** delle risorse di un computer, non necessariamente tutte.
- Non è richiesto un intero OS, basta un'applicazione.
- Si usano **organizzazioni virtuali** (simili ai gruppi in Unix) per gestire i permessi.

1.4.4 Cloud Computing

Il cloud computing si basa sulla **virtualizzazione** per offrire risorse on-demand. L'idea centrale è che l'utente non sa esattamente quale infrastruttura fisica sta usando – c'è uno strato intermedio di astrazione.

Le risorse sono offerte tramite **SLA (Service Level Agreements)**, contratti che definiscono le garanzie di prestazione. I modelli di servizio principali sono:

Acronimo	Nome	Descrizione
IaaS	Infrastructure as a Service	Si noleggia l'infrastruttura (VM, storage, rete)
PaaS	Platform as a Service	Si noleggia la piattaforma (OS, DB, middleware)
SaaS	Software as a Service	Si usa direttamente il software (Gmail, Office 365)

1.4.5 Sistemi Pervasivi Distribuiti

Concetto relativamente recente. Il sistema si integra nell'ambiente dell'utente in modo quasi invisibile. Ogni aspetto della vita è connesso alla rete. Si divide in:

- **Ubiquitous computing**: il sistema è sempre presente e disponibile.
- **Mobile computing**: il sistema si sposta fisicamente con l'utente.
- **Reti di sensori (e attuatori)**: i nodi interagiscono con il mondo fisico – leggono dati dai sensori o agiscono sull'ambiente tramite attuatori.

1.5 Server Cluster

In un server cluster, più server cooperano per offrire un unico servizio al cliente. Il cliente non sa con quale specifica macchina sta comunicando – vede solo il servizio.

Le principali strategie per gestire le richieste sono:

- **Transport Layer Switching**: il *front-end* passa la richiesta TCP a uno dei server usando qualche metrica (es. round-robin, carico corrente).
- **Context-aware distribution**: il front-end instrada la richiesta in base al contenuto effettivo della richiesta e ai dati già presenti su ciascun server.

In entrambi i casi, lato client tutto è **trasparente**.

1.6 Virtualizzazione

La virtualizzazione è la tecnica per simulare un comportamento (hardware o software) diverso da quello reale. L'utente ha l'illusione di usare un sistema diverso da quello fisicamente presente.

Vantaggi principali:
- Maggiore **portabilità** del codice.
- Più facile **manutenzione e aggiornamento**.
- Gestione semplificata dei guasti di componenti specifici.

Esistono tre tipi di virtualizzazione:

Tipo	Descrizione
Process VM	Virtualizzazione a livello di processo (es. JVM di Java)
Native VMM	Hypervisor che gira direttamente sull'hardware (es. VMware ESXi)
Hosted VMM	Hypervisor che gira sopra un OS ospite (es. VirtualBox, VMware Workstation)

Nel cloud computing, la virtualizzazione (tipicamente IaaS) permette di isolare le risorse di ciascun utente – ognuno usa solo ciò per cui paga.

Migrazione del codice

La **code migration** serve per:
- Sfruttare meglio i server.
- Eseguire i calcoli più vicino ai dati.
- Evitare installazioni ripetute.

Modello	Mobilità del codice	Mobilità dell'esecuzione
Client-server	No	No
Remote evaluation	No	Sì (es. Jupyter Notebook)
Code-on-demand	Sì	No
Mobile agents	Sì	Sì

Migrazione di Virtual Machine

È la forma moderna dei mobile agent. Le strategie principali sono:
- **Stop-and-restart**: si ferma la VM e la si riavvia sul nuovo host.
- **Pre-copy**: si trasferisce la memoria mentre la VM è ancora in esecuzione, poi si ferma e si riavvia.
- **Demand paging**: si avvia la VM sul nuovo host e le pagine di memoria si caricano on-demand.

> La migrazione richiede tempo e risorse, quindi va pianificata con attenzione.

CAPITOLO 2 – Sistemi Cyber-Fisici e System of Systems

2.1 Cos'è un Sistema Cyber-Fisico (CPS)?

Un **Cyber-Physical System (CPS)** è un sistema che combina componenti computazionali (la parte *cyber*: processori, software, reti) con processi fisici (la parte *physical*: sensori, attuatori, leggi della fisica). L'interazione tra le due parti è continua e bidirezionale.

- **Cyber**: elaborazione e comunicazione delle informazioni.
- **Physical**: il mondo reale, governato dalle leggi della fisica.
- **Umano**: il fattore umano può essere presente o assente.

Esempi: automobili autonome, sistemi ferroviari, droni, smart grid, impianti industriali automatizzati.

2.2 Da Sistemi Monolitici a System of Systems

Sistemi Monolitici (mainframe era)

I primi computer erano concepiti come sistemi chiusi e non aggiornabili. Un **sistema monolitico** ha tutte le funzionalità integrate in un unico blocco:

- Non è modulare.
- È difficile da aggiornare (sia hardware che software).
- Non è pensato per interagire con altri sistemi.

> Nota: esistono ancora componenti monolitici moderni. Il kernel di Linux, ad esempio, è monolitico – ma Linux nel complesso rimane modulare perché si può estendere il kernel con moduli aggiuntivi.

Il paradigma moderno: Systems of Systems (SoS)

Un **System of Systems (SoS)** è un'integrazione di un numero finito di sistemi costituenti (*Constituent Systems*, CS), che sono **indipendenti** e **operabili autonomamente**, e che

vengono interconnessi per un periodo di tempo per raggiungere un obiettivo comune di livello superiore (definizione di Jamshidi).

Punto chiave: ciascun CS può funzionare da solo, ma insieme producono qualcosa di più del semplice insieme delle parti.

Esempi di SoS:

- **Elicottero**: CPS con scope predefinito, test di accettazione classico, guasti accettabili.
- **Sistema ferroviario**: CPS moderno, scope definito (molti servizi, un obiettivo), test sul campo, guasti quotidiani da gestire.
- **Automobili nel traffico**: ogni veicolo vuole andare da A a B; in un ingorgo devono interagire. A volte trattati come SoS anche con interazione cyber minima.
- **Alexa e casa intelligente**: Alexa integra diversi sistemi domestici. Un guasto al hub centrale può causare problemi reali (luci accese di notte, finestre aperte).

2.3 Caratteristiche Distintive dei SoS

Confronto: Sistemi Monolitici vs. System of Systems

Caratteristica	Sistema Monolitico	System of Systems
Scope	Fisso e noto	Incerto o variabile
Sincronizzazione clock	Interna	Esterna (GPS, NTP)
Struttura	Gerarchica rigida	Networked (concettualmente gerarchica)
Requisiti	Fissi	In continua evoluzione
Evoluzione	Version control	Non coordinata (ogni CS evolve per conto suo)
Testing	Fasi di test definite	Continuo (si testa sul campo)
Tecnologia	Nota e fissa	Eterogenea e variabile
Guasti	Eccezionali	Normali (da anticipare e gestire)
Controllo	Centralizzato	Autonomo (ogni CS ha una certa libertà)
Emergenza	Trascurabile	Fondamentale
Sviluppo	Processo definito	Ogni CS si sviluppa in modo indipendente

Peculiarità organizzative dei SoS

- Lo **scope** è spesso incerto: non si sa esattamente dove finisce un SoS.
- Non esiste necessariamente un **acceptance test finale** nel senso classico.
- I **guasti** devono essere gestiti sempre come eventi normali, non come eccezioni.

2.4 Tipologie di System of Systems

I SoS si classificano in base a **chi controlla i CS** e **quanto è centrale il coordinamento**:

Tipo	Controllo centrale	Ownership	Esempi
Directed	Sì	Unico proprietario	Automobile, satellite
Acknowledged	Cooperativo con regole	Proprietari indipendenti ma coordinati	Sistema dell'aviazione
Collaborative	Volontario	Indipendenti	Rete ATM, Bitcoin miners
Virtual	Nessuno	Nessuna struttura centrale	Internet

> Il caso **Virtual** è il più estremo: non c'è scopo centrale condiviso né coordinamento formale. Internet è praticamente l'unico esempio puro.

2.5 Concetti Fondamentali: Dominio, Sistema, Ambiente

- **Dominio**: l'insieme di entità e relazioni che modelliamo. Aggiungiamo al dominio tutto ciò che è necessario per rappresentare una visione selezionata del mondo reale.

- **Sistema**: un'entità capace di modificare l'ambiente in cui opera. È tipicamente sensibile al passare del tempo.

- **Ambiente**: il contesto in cui il sistema opera (es. le strade sono l'ambiente di un'automobile).

- **Confini (Boundaries)**: ogni sistema ha confini che delimitano cosa è "dentro" e cosa è "fuori". I confini non sono sempre facili da definire:
- Un'automobile deve percepire anche le corsie adiacenti, non solo la propria.
- Se un pedone attraversa in modo imprevedibile, l'auto deve anticiparlo *prima* che entri nella sua corsia.
- I confini possono non essere stabili: un'auto a bassa velocità ha bisogno di meno informazioni ambientali rispetto a una ad alta velocità.

- **Architettura del sistema**: il progetto complessivo del sistema. Definisce la struttura generale (con più o meno dettaglio a seconda delle esigenze).

2.6 Gerarchia Multi-livello nei SoS

I SoS hanno tipicamente una **struttura gerarchica multi-livello** (o ricorsiva):
- A livello **macro**: il sistema è visto nella sua totalità.
- A livello **micro**: ogni sottosistema può essere suddiviso in ulteriori sottosistemi.
- La ricorsione termina quando i dettagli interni non sono più rilevanti per lo scopo dell'analisi.

Esempio: analizzo il sistema ferroviario.
- Se sto pianificando l'infrastruttura, non mi interessa il codice dei singoli dispositivi.
- Se sto sviluppando il software di un sistema di segnalamento, devo andare nel dettaglio del codice.

Un concetto correlato è quello di **Holon e Holarchy**:
- Un **holon** è un'entità che funziona sia come *tutto* (a livello macro) sia come *parte* (a livello micro).
- Una **holarchy** è un sistema strutturato di holons, dove ogni livello è composto da holons che formano holons di livello superiore.
- Le interazioni in una holarchy sono **verticali** (relazioni di interazione tra livelli).
- È come un'orchestra: il direttore guida i musicisti, ma non suona loro le note una per una.

2.7 Emergenza

Definizione

L'**emergenza** è uno dei concetti più affascinanti (e complessi) dei SoS. Nasce dalla citazione attribuita ad Aristotele: *"Il tutto è più della somma delle parti"*.

> Un fenomeno è definito **emergente** se e solo se è *nuovo rispetto ai fenomeni non relazionali di ciascuna delle sue parti proprie a livello micro*.

In parole semplici: quando i sistemi interagiscono tra loro, possono dare origine a proprietà o comportamenti che nessun singolo sistema avrebbe da solo.

Emergenza positiva e negativa

L'emergenza può essere:

- **Positiva**: la collaborazione tra CS produce qualcosa di utile e desiderato (es. la rete ATM offre all'utente la possibilità di prelevare da qualsiasi sportello, anche se ogni ATM funziona da solo).
- **Negativa**: la collaborazione produce effetti indesiderati e spesso imprevedibili.

Esempi di emergenza negativa

- **Deadlock**: guardando i singoli processi non si può prevedere con certezza che si verificherà un deadlock. Il deadlock è emergente perché nasce dall'interazione tra processi che individualmente non hanno questo problema.
- **Thrashing**: in un sistema operativo, il thrashing si verifica quando il SO passa più tempo a gestire la memoria virtuale che a eseguire processi.
- **Crollo del mercato azionario**: gli investitori individuali prendono decisioni razionali, ma l'effetto aggregato può causare un collasso.

> Nota: spesso un fenomeno ci sembra "emergente" perché non siamo in grado di prevederlo, non perché sia intrinsecamente imprevedibile. L'emergenza è talvolta un prodotto della nostra ignoranza.

Bridge laws e downward causation

Per descrivere l'emergenza tra livelli diversi si usano le **bridge laws** (leggi di ponte): regole che descrivono come i fenomeni a un livello inferiore danno origine a proprietà a un livello superiore.

Esempio: atomi → molecole → cellule → sostanze medicinali.

La **downward causation** descrive il fenomeno per cui il macro-livello impone vincoli ai singoli componenti del micro-livello. Nel caso del deadlock: lo scheduling (macro) impone vincoli ai processi (micro) che portano al deadlock.

2.8 Controllo nei SoS

In un SoS il livello di controllo può variare da assente a totale. Le due forme principali sono:

- **Autorità dall'esterno**: il controllo è imposto da un'entità esterna (governo, azienda, standard internazionale).
- **Autorità dall'interno**: il controllo è esercitato dai componenti al livello superiore della gerarchia (macro) su quelli al livello inferiore (micro).

L'autorità è fondamentale per garantire l'emergenza positiva: senza una qualche forma di coordinamento, i CS non collaborano abbastanza da produrre comportamenti emergenti utili.

2.9 Stigmergia

La **stigmergia** è un concetto mutuato dalla biologia (lo studio delle formiche). Descrive il coordinamento **indiretto** tra agenti tramite modifiche all'ambiente.

Origine biologica: le formiche lasciano tracce di feromoni nell'ambiente. I percorsi più battuti diventano più intensi, guidando le altre formiche verso le rotte più efficienti. Nessuna formica decide centralmente quale percorso usare.

In un CPS: i sistemi lasciano tracce nell'ambiente (fisico o informazionale) che influenzano il comportamento degli altri sistemi. Esempi:

- **Smart car**: un'ambulanza emette segnali che le auto intelligenti rilevano per farsi da parte.
- **Smart city**: i sensori di traffico modificano i tempi dei semafori, influenzando il comportamento dei veicoli.
- **Sistemi multi-agente**: agenti che modificano lo stato dell'ambiente per comunicare indirettamente.

La stigmergia è strettamente legata al concetto di **entourage** di un CS: l'area fisica intorno al CS dove possono avvenire interazioni stigmergiche. Quando gli entourage di due CS si sovrappongono, è possibile lo scambio di segnali stigmergici.

2.10 Dinamicità ed Evoluzione

I CPS e i SoS devono affrontare cambiamenti a due scale temporali diverse:

- **Dinamicità** (breve termine): adattamento rapido alle variazioni dell'ambiente o dei requisiti operativi.

- **Evoluzione** (lungo termine): cambiamenti strutturali al sistema nel tempo. Può essere:
- **Gestita (Managed evolution)**: si aggiorna un componente in modo pianificato, sapendo in anticipo l'impatto sull'intero SoS.
- **Non gestita (Unmanaged evolution)**: qualcuno modifica un componente su cui tu dipendi senza che tu ne sia informato.

L'evoluzione può anche essere:

- **Locale**: non cambia le interfacce esterne (RUI), ottimizza solo il CS internamente.
- **Globale**: cambia le RUI, quindi può impattare altri CS.

La **Riconfigurabilità** è la capacità di un sistema di mantenere il servizio durante guasti o aggiornamenti, modificando la propria struttura o configurazione.

2.11 Dipendabilità e Sicurezza

Dipendabilità (Dependability)

La **dipendabilità** è la capacità di un sistema di fornire un servizio affidabile. Gli attributi principali sono:

Attributo	Descrizione
Availability	Il sistema è disponibile quando richiesto
Reliability	Il sistema funziona correttamente per un periodo dato
Safety	Il sistema non causa danni in caso di guasto
Confidentiality	Le informazioni sono accessibili solo agli autorizzati
Integrity	I dati non vengono modificati in modo non autorizzato
Maintainability	Il sistema può essere riparato o aggiornato facilmente

Per raggiungere la dependability usano quattro strategie:

- **Fault Prevention**: prevenire i guasti in fase di progetto.
- **Fault Tolerance**: tollerare i guasti a runtime.
- **Fault Removal**: rimuovere i guasti attraverso test e verifica.
- **Fault Forecasting**: prevedere i guasti e le loro conseguenze.

Sicurezza (Security) – Triade CIA

- **Confidentiality**: solo chi è autorizzato può accedere alle informazioni.
- **Integrity**: i dati non vengono alterati in modo non autorizzato.
- **Availability**: il servizio è disponibile agli utenti autorizzati.

> **Esempi storici di fallimenti**:
> - **Ariane 5**: crash per un overflow causato dal riuso di software di Ariane 4.

> - **Mars Climate Orbiter**: perso per una mancata conversione da unità imperiali a metriche.
> - **Knight Capital**: perdita di 440 milioni di dollari in 45 minuti per un bug nel software di trading.
> - **Missile Patriot**: mancato abbattimento per un errore di arrotondamento nei calcoli temporali.

Verifica vs. Validazione

- **Verification**: stiamo costruendo il sistema *nel modo giusto*? (rispettiamo le specifiche?)
- **Validation**: stiamo costruendo *il sistema giusto*? (il sistema fa quello che l'utente vuole davvero?)

CAPITOLO 3 – Gestione del Tempo nei Sistemi Distribuiti e CPS

3.1 Perché il Tempo è un Problema?

In un sistema distribuito o in un SoS, ogni nodo ha il proprio orologio fisico interno. Questi orologi sono implementati tramite un **oscillatore** (un cristallo al quarzo) e un **registro contatore**: ogni volta che l'oscillatore completa un periodo, viene generato un evento (*tick*) e il registro viene incrementato.

Il problema fondamentale è che gli oscillatori **non sono perfetti**: tendono a desincronizzarsi nel tempo a causa del *drift* (deriva). Due macchine avviate con lo stesso orario esatto dopo qualche tempo mostreranno orari diversi, anche se di pochissimo.

Nei sistemi distribuiti questo crea problemi seri:

- Non è possibile stabilire quale evento è avvenuto prima tra due nodi diversi.
- I protocolli di comunicazione si basano spesso su timeout temporali.
- Le operazioni distribuite (es. transazioni) devono essere ordinate nel tempo.

3.2 Concetti Base sul Tempo

Definizioni fondamentali

Termine	Definizione

Tempo	Quantità fisica continua e misurabile in cui gli eventi si susseguono dal passato al futuro
Timeline	Linea densa che indica la progressione del tempo. Su di essa si identificano *istanti* ed *eventi*
Istante	Un punto preciso sulla timeline
Evento	Un'occorrenza che si verifica in un istante preciso
Ciclo	Sequenza temporale di eventi che arriva a uno stato finale da cui può essere riavviata
Periodo	Un ciclo con una durata **costante** tra lo stato iniziale e quello finale

> Attenzione: la distinzione tra ciclo e periodo è importante. Un *ciclo* non ha durata fissa, un *periodo* sì. I sistemi di controllo real-time si basano su periodi precisi.

Sistemi periodici

Un **sistema periodico** è un sistema il cui comportamento temporale è strutturato in una sequenza di periodi. Le applicazioni di controllo si basano fortemente su questa struttura.

Esempio pratico – Bus CAN con ECU:
Immaginiamo un bus di comunicazione (es. in un'automobile) con più ECU (Electronic Control Unit) collegate. Senza un clock condiviso, ogni ECU comunicherebbe in momenti casuali, generando collisioni. Con un sistema periodico:

- Si definisce un *chunk* di tempo (= periodo) suddiviso in *slot*.
- Ogni ECU può trasmettere solo nel suo slot dedicato.
- Le collisioni sono azzerate.
- Anche se un'ECU non ha dati da mandare, trasmette un *pacchetto vuoto* per segnalare che è ancora attiva.

3.3 Standard Temporali

Il secondo fisico è lo stesso in tutti gli standard comunemente usati. Il punto critico è con quale riferimento si *allinea* questo secondo.

Standard	**Descrizione**
UTC (Universal Time Coordinated)	Basato sulla rotazione della Terra, che non è costante. Vengono aggiunti periodicamente "leap seconds" per compensare.
TAI (International Atomic Time)	Basato su orologi atomici ultra-precisi. Non tiene conto della rotazione terrestre.

GPS Time	Usa il TAI come riferimento. Distribuito dai satelliti GPS. Non è sempre allineato con UTC (differenza di circa 18 secondi).

> Il GPS usa il **TAI** (non UTC), che non include i *leap seconds*. Questo può causare disallineamenti di circa 18 secondi rispetto all'UTC nei sistemi che non gestiscono correttamente questa differenza.

3.4 Tipi di Orologi

Orologio di riferimento (Reference Clock)

Un orologio allineato con il TAI con una granularità più piccola di qualsiasi durata di interesse. È il "metro di riferimento" ideale.

Orologio coordinato (Coordinated Clock)

Un orologio sincronizzato entro certi limiti con un reference clock che si trova fisicamente altrove. Non può essere perfettamente uguale al reference clock per via dei costi e dei ritardi di comunicazione. Per questo si definisce un **intervallo di confidenza**: uno scarto massimo accettabile per il sistema.

Esempio: nel bus CAN dell'esempio precedente, si definisce una *grace period* (finestra di tolleranza) che permette ai pacchetti di arrivare leggermente in anticipo o ritardo rispetto al loro slot, senza essere considerati in errore.

3.5 Metriche di Qualità degli Orologi

Per descrivere quanto un orologio si discosta dal reference clock si usano tre metriche fondamentali:

Metrica	Descrizione
Drift	Misura della qualità della frequenza dell'orologio rispetto al reference clock. Indica quanto velocemente guadagna o perde tempo (espresso in ppm – parts per million).

Precision (π)	*Internal clock synchronization* – la massima distanza accettata tra due orologi nel sistema.
Accuracy (α)	*External clock synchronization* – la massima distanza accettata tra l'orologio interno e una funzione di riferimento esterna.
Offset	La distanza istantanea tra il clock ideale (perfetto) e il clock reale.

> Il drift può essere molto piccolo ma cumulativo nel tempo. Correggere un drift molto piccolo può talvolta peggiorare le cose anziché migliorarle (introduce oscillazioni).

Assunzioni di sistema:
- Ogni sistema ha un drift massimo garantito.
- Si determina l'accuracy e la precision massima accettabili.
- La *grace period* del bus CAN è dimensionata sulla precision supposta del sistema.

3.6 Come si Sincronizzano gli Orologi?

3.6.1 Sincronizzazione tramite GPS

Il GPS è basato su circa **20 satelliti** che orbitano in modo da essere sempre visibili da ogni punto della Terra. Il meccanismo di sincronizzazione funziona così:

- Il satellite conosce il **TOF** (Time of Flight): il tempo impiegato dal segnale per raggiungere il ricevitore.
- Conoscendo il TOF si sa di quanto compensare il ritardo di propagazione.
- I satelliti si comunicano tra loro per determinare il loro reference clock (basato su orologi atomici a bordo).
- Il ricevitore usa i segnali di almeno 4 satelliti per calcolare posizione e tempo con alta precisione.

3.6.2 Sincronizzazione tramite NTP

NTP (Network Time Protocol) è il protocollo usato dai computer in rete per sincronizzare gli orologi. Si basa su una **gerarchia di livelli** (*stratum*):

- **Stratum 0**: orologio atomico o GPS (fonte primaria).
- **Stratum 1**: server direttamente connesso allo Stratum 0.
- **Stratum 2**: server sincronizzato tramite Stratum 1.
- E così via...

Come calcola l'offset:

- Il client invia un messaggio al server al tempo **T1** (misurato dal client).
- Il server riceve al tempo **T2**, risponde al tempo **T3** (entrambi misurati dal server).
- Il client riceve la risposta al tempo **T4**.
- **Round-trip delay**: $\delta = (T4 - T1) - (T3 - T2)$
- **Offset stimato**: $\theta = ((T2 - T1) + (T3 - T4)) / 2$

> Per la formula matematica completa e gli esempi numerici, fare riferimento alle slide del corso.

3.7 Dati, Informazione e Stato

Dato vs. Informazione

- **Dato**: un artefatto, un pattern creato per uno scopo specifico. È la rappresentazione "grezza" di qualcosa (es. i bit in un file).
- **Informazione**: qualcosa che è in grado di rappresentare una cosa vera nel mondo reale. Il dato diventa informazione quando il ricevitore riesce a estrarne significato.
- **Metadata**: informazione *sul* dato (es. data di creazione, autore, dimensione di un file).

Stato di un sistema

Lo **stato** di un sistema è la totalità delle informazioni del passato che possono influenzare il comportamento futuro del sistema.

Concetto chiave: lo stato senza il concetto di tempo è privo di significato. Lo stato di un sistema è sempre riferito a un istante temporale preciso.

Lo **spazio degli stati** (*state space*) è la totalità dei possibili valori che le variabili di stato del sistema possono assumere.

3.8 Vista Event-Based vs. Vista State-Based

Esistono due approcci fondamentalmente diversi per monitorare e reagire ai cambiamenti in un sistema:

Vista Event-Based (basata sugli eventi)

I valori delle variabili vengono aggiornati *quando si verifica un evento*. Il sistema non fa nulla finché non arriva un trigger.

Problema: se il numero di eventi è illimitato (o imprevedibile), il numero di possibilità da analizzare diventa anch'esso illimitato.

Esempio: un sistema di acquisto di un'auto. Quando qualcuno acquista l'auto, lo stato cambia da "in vendita" a "venduta".

Vista State-Based (basata sullo stato / campionamento)

Si basa su una tecnica chiamata **sampling** (campionamento). Si osservano i valori delle variabili di stato in *istanti di osservazione* periodici. L'intervallo di campionamento deve essere scelto con cura: troppo piccolo genera troppo overhead, troppo grande perde informazioni.

Esempio: una telecamera analizza lo stato di un semaforo a intervalli regolari per determinare se è rosso o verde.

Quale usare?

Nessuno dei due è superiore all'altro in assoluto – hanno casi d'uso diversi:

Situazione	**Approccio consigliato**
Contatore assi in una ferrovia (rileva il momento esatto del passaggio)	**Event-based**
Semaforo per treni controllato automaticamente da una telecamera	**State-based**

> Un sistema può essere sia event-based che state-based contemporaneamente.

3.9 Sistemi Time-Triggered vs. Event-Triggered

Sistemi Time-Triggered (TT)

Un sistema **time-triggered** è un sistema in cui le azioni vengono scatenate dal *passare del tempo*. Ogni componente è autorizzato ad agire in base a un calendario temporale predefinito.

- **Execution time**: il tempo necessario per eseguire un'azione specifica.
- **WCET (Worst Case Execution Time)**: il tempo di esecuzione nel caso peggiore (sistema completamente carico). Usato per dimensionare i sistemi real-time.

Sistemi Event-Triggered (ET)

Un sistema **event-triggered** è un sistema in cui le azioni vengono scatenate da *eventi esterni* (non dal solo passare del tempo).

> Un sistema può combinare entrambi gli approcci: alcune azioni sono periodiche (TT), altre reattive (ET).

Sistemi Real-Time

Un **sistema real-time** è un sistema in cui ogni azione deve essere completata entro una *deadline* specifica. Per progettare un sistema real-time correttamente:
- Si valuta il **WCET** di ogni operazione.
- Si fissa la deadline tenendo conto del WCET più un margine di sicurezza.

Nei sistemi CPS, il rispetto delle deadline è spesso critico per la sicurezza fisica (es. sistemi frenanti ABS, autopiloti).

3.10 Comportamento di un Sistema

Una **funzione** è una mappatura da un insieme di valori di input a un insieme di valori di output.

Il **comportamento** di un sistema è la sequenza temporizzata degli effetti delle azioni di input e output osservabili all'interfaccia del sistema.

Un sistema può essere:
- **Deterministico**: lo stesso input produce sempre lo stesso output.
- **Non-deterministico**: lo stesso input può produrre output diversi (tipico con la concorrenza).

3.11 Comunicazione e Tipi di Messaggi

Un **messaggio** è una struttura dati creata per scambiare informazioni tra due sistemi. I messaggi si differenziano principalmente per le *garanzie di consegna* e i *requisiti temporali*:

Tipo	Garanzia di consegna	Vincolo temporale	Esempio
Datagram	Nessuna	Nessuno	UDP
PAR-Message	Sì (delivery garantita)	Non definito (alto jitter)	TCP
TT-Message	Sì	Sì (deadline + periodicità)	TDMA

Message jitter: la differenza tra il momento in cui un messaggio è *atteso* e il momento in cui è *effettivamente ricevuto*. Nei PAR è tipicamente alto, nei TT è molto basso.

Temporal error detection:
- Nei **PAR-Message**: l'errore temporale viene rilevato dal *mittente* (che attende un ACK entro un timeout).
- Nei **TT-Message**: l'errore temporale viene rilevato dal *ricevitore* (che sa esattamente quando aspettarsi il messaggio).

CAPITOLO 4 – Interfacce nei Sistemi Cyber-Fisici

4.1 Cos'è un'Interfaccia in un SoS?

Un'**interfaccia** in un System of Systems è un punto in cui un componente specifico del sistema interagisce con un altro componente o con l'ambiente esterno. Le interfacce sono uno dei concetti più importanti nell'ingegneria dei SoS perché:

- Permettono di **separare** le responsabilità tra sistemi diversi.
- Devono essere **identificate, specificate e standardizzate** con cura.
- Agiscono da **firewall di complessità**: nascondono i dettagli interni di un CS agli altri.

> L'identificazione e la standardizzazione delle interfacce è uno dei passi più critici nella progettazione di un SoS. Una cattiva interfaccia può bloccare l'evoluzione futura dell'intera architettura.

4.2 Itom: l'Unità di Scambio

Il termine **Itom** (Information Technology Object Messages) indica l'unità base di informazione scambiata tra i sistemi attraverso le interfacce dell' Informational Interface Level (vedi cap. 4.6).

Ogni Constituent System (CS) elabora due tipi di itom:

- **Itom ricevuti**: provengono dallo spazio cyber (da altri CS) o da sensori fisici che osservano l'ambiente fisico circostante (*entourage*). Es: in una sala server, l'unico itom rilevante potrebbe essere la temperatura ambiente.

- **Itom prodotti**: vengono generati dal CS e inviati verso l'ambiente fisico (tramite attuatori) o verso lo spazio cyber (verso altri CS).

Esempio reale – Cockpit di un aereo:
- *Itom ricevuti*: messaggi dai sensori di bordo verso l'autopilota e il cockpit, messaggi dalla torre di controllo a terra.
- *Itom prodotti*: messaggi verso la torre sulla posizione attuale, comandi verso gli attuatori (flap, motori, ecc.).

4.3 Modello Ambientale ed Entourage

L'**environmental model** (modello ambientale) è una rappresentazione formale dell'ambiente in relazione al sistema. In questo modello vengono rappresentate le interrelazioni tra le variabili dell'ambiente e quelle del sistema.

Il concetto di **entourage** di un CS indica l'area fisica circostante il CS in cui possono avvenire interazioni rilevanti. È la zona di "influenza diretta" del sistema.

Sovrapposizione degli entourage

Nel mondo reale è normale che gli entourage di CS diversi si sovrappongano. Questa sovrapposizione è fondamentale per la **stigmergia**: quando due CS hanno entourage sovrapposti, possono scambiarsi itom attraverso canali fisici o informazionali indiretti.

Sintesi del modello:
- Ogni CS cyber-fisico si trova in un ambiente fisico e ha un entourage fisico.
- La sovrapposizione degli entourage abilita canali stigmergici e quindi lo scambio di itom.
- L'obiettivo è concettualizzare le interazioni time-sensitive nei CPSoS per poter definire layer di interfaccia e produrre un modello di design efficace.

4.4 Layer di Interfaccia

Dividere le interfacce in **layer** (strati) permette di discutere e specificare separatamente le proprietà di ciascun livello. I tre layer principali nei CPSoS sono:

Layer	Nome	Cosa gestisce
Layer 1	Cyber-Physical Interface	Interazione tra componenti cyber e il mondo fisico (sensori/attuatori)
Layer 2	Informational Interface	Astrazione delle informazioni dal layer fisico
Layer 3	Service Interface	Servizi composti, architetture orientate ai servizi (SoA)

Ogni layer costruisce un'astrazione sopra il precedente, nascondendo la complessità e offrendo un'interfaccia più pulita al layer superiore.

4.5 Layer 1 – Cyber-Physical Interface

Questo è il layer più basso e più vicino al mondo fisico. In questo layer le informazioni sono rappresentate come **data item** grezzi provenienti da sensori o diretti verso attuatori.

I due canali di comunicazione principali a questo livello sono:

- **RUMI** (*Relied Upon Message Interface*): canale che permette la comunicazione tra la parte cyber e la parte fisica del sistema. È il canale attraverso cui i sensori inviano misurazioni al processore e il processore invia comandi agli attuatori.

- **RUPI** (*Relied Upon Physical Interface*): canale che permette la comunicazione tra le parti cyber di sistemi diversi attraverso il mezzo fisico. È fondamentale per la realizzazione della stigmergia tra CS con entourage sovrapposti.

> Nota: esiste anche un canale **RUHI** (*Relied Upon Human Interface*) per la comunicazione con gli operatori umani, spesso trascurato ma importante nei sistemi semi-automatici.

In questo layer avviene il primo scambio di itom: l'informazione grezza viene raccolta e preparata per essere raffinata nel layer superiore.

4.6 Layer 2 – Informational Interface

Il layer informazionale è il luogo in cui le informazioni vengono **astratte e raffinate**. Il tempo è un fattore cruciale a questo livello: gli itom devono essere elaborati rispettando vincoli temporali precisi.

Le attività principali a questo layer sono:

- **Rimozione dei dettagli**: le informazioni vengono filtrate e semplificate, eliminando i dati non necessari per i livelli superiori.
- **Flussi di informazione context-independent**: le informazioni sono rese indipendenti dal contesto fisico specifico.
- **Specifica e raffinamento degli itom**: gli itom vengono definiti esplicitamente e raffinati prima di essere passati al layer superiore.

Esempio – Freno di emergenza:
Una macchina deve effettuare una frenata d'emergenza. A livello fisico (Layer 1) i sensori rilevano l'ostacolo. A livello informazionale (Layer 2), la macchina trasmette ad altri veicoli nelle vicinanze un segnale che li avvisa di rallentare. L'elaborazione dell'ostacolo da parte del singolo veicolo rimane al Layer 1, mentre la comunicazione tra veicoli avviene al Layer 2.

Canali indiretti e Environmental Constituent System: usando canali indiretti (stigmergici) a questo layer, più SoS possono creare un *Environmental Constituent System* che realizza l'ambiente comune di interazione tra i vari CS.

4.7 Layer 3 – Service Interface

A questo livello non interessa più come funzionano internamente i CS, ma solo **quali servizi offrono**. È il layer dedicato all'architettura orientata ai servizi (**SoA – Service Oriented Architecture**).

Principi della SoA

- Si parte dai *servizi* stessi come unità fondamentali di progettazione.
- Ogni servizio è considerato **indipendente** o **debolmente accoppiato** con gli altri.
- Ogni servizio dovrebbe essere piccolo e focalizzato su una singola responsabilità.

Componenti della SoA

In una SoA esistono due attori principali:
- **Service Provider**: chi offre il servizio.
- **Service Consumer**: chi utilizza il servizio.

I due standard tecnici principali per l'implementazione sono:
- **REST** (*Representational State Transfer*): basato su HTTP, leggero e scalabile.
- **SOAP** (*Simple Object Access Protocol*): basato su XML, più rigido ma con maggiori garanzie formali.

Sopra questi componenti si può aggiungere:
- **Orchestratore**: un componente centralizzato che decide l'ordine in cui i servizi vengono invocati.
- **Coreografia**: in alternativa all'orchestratore, i servizi si coordinano autonomamente senza un punto centrale.

Interface Service Specification

È un insieme di **metriche di qualità** che permettono di valutare la qualità del servizio offerto. I service provider pubblicano le proprie capacità attraverso un **SLA (Service Level Agreement)**: un contratto che definisce le garanzie di prestazione, disponibilità e comportamento del servizio.

4.8 RUI – Relied Upon Interface

Una **RUI** (*Relied Upon Interface*) è un'interfaccia su cui altri componenti del sistema *fanno affidamento*. È la tipologia di interfaccia più importante in un SoS. É presente a tutti e 3 i livelli definiti sopra.

Strategia di connessione RUI

Quando si lavora con le RUI bisogna considerare la **RUI Connecting Strategy**:
- Si cercano le connessioni disponibili nell'ambiente.
- Si stabilisce la connessione.
- La connessione viene interrotta quando non è più necessaria.

Ruoli della RUI

Una RUI svolge molteplici funzioni nel SoS:

Ruolo	Descrizione
System boundary	Definisce il confine del sistema verso l'esterno
Complexity firewall	Nasconde la complessità interna del CS
Information transfer	Veicola il trasferimento di informazioni tra CS
Emergence	Abilita l'emergenza attraverso le interazioni tra CS
Dynamicity	Supporta l'adattamento dinamico del sistema
Evolution	Permette l'evoluzione controllata del sistema

Tipi di RUI

Sigla	Nome completo	Funzione
RUMI (Layer 1)	Relied Upon Message Interface	Comunicazione tra cyber e fisico dello stesso CS
RUPI (Layer 1)	Relied Upon Physical Interface	Comunicazione fisica tra CS diversi (stigmergia)
RUS (Layer 3)	Relied Upon Service	Servizio con SLA garantito
TSI	Time Synchronization Interface	Sincronizzazione del tempo globale
D-Interface	Diagnostic Interface	Diagnostica del sistema

> Nota: al layer cyber-physical, le diverse componenti della RUI interagiscono tra loro e spesso si connettono a certe tipologie di sensori, come i satelliti GPS per la sincronizzazione dell'orologio. Fare riferimento alle immagini nelle slide per una visione più approfondita.

4.9 Gestione dell'Evoluzione dell'Interfaccia

Le interfacce nei SoS devono evolvere nel tempo. Si distinguono due tipi principali:

Evoluzione gestita (Managed Evolution)

Il CS viene aggiornato in modo che le modifiche **non cambino il comportamento del sistema** verso l'esterno, oppure le modifiche sono pianificate e comunicate a tutti i CS che dipendono da quell'interfaccia.

Evoluzione non gestita (Unmanaged Evolution)

Qualcun altro aggiorna un CS o un'interfaccia da cui il nostro sistema dipende, **senza preavviso**. Questo è il caso tipico nei SoS aperti, dove i CS appartengono a proprietari diversi.

L'evoluzione può anche essere classificata in:

- **Evoluzione locale**: non cambia le RUI; ottimizza solo l'interno del CS. Gli altri CS non sono impattati.
- **Evoluzione globale**: cambia le RUI. Gli altri CS che dipendono da quelle interfacce devono essere aggiornati di conseguenza.

CAPITOLO 5 – Coordinamento Distribuito

5.1 Introduzione al Problema del Coordinamento

Il coordinamento distribuito è uno dei problemi fondamentali dei sistemi distribuiti. In un sistema centralizzato, un singolo nodo può coordinare tutte le operazioni usando il proprio orologio e la propria memoria. In un sistema distribuito, invece, non esiste un clock globale né una memoria condivisa affidabile: ogni nodo ha la propria visione locale del mondo.

I problemi principali che il coordinamento distribuito deve risolvere sono:

- **Event-ordering**: in che ordine sono avvenuti gli eventi su nodi diversi?
- **Mutual exclusion**: come garantire che solo un processo alla volta acceda a una risorsa condivisa?
- **Atomicity**: come garantire che un'operazione avvenga interamente o non avvenga affatto?
- **Deadlock**: come rilevarlo e risolverlo senza un coordinatore centrale?
- **Consensus**: come far accordare tutti i nodi su un singolo valore?

> Molti di questi argomenti sono trattati anche nel corso parallelo di Distributed Systems (DPWIM). In questo corso ci concentriamo soprattutto sull'**ordinamento degli eventi** e sul **consensus**, con particolare attenzione ai sistemi cyber-fisici.

5.2 Il Problema dell'Ordinamento degli Eventi

Perché è difficile?

In un sistema distribuito ogni macchina ha il proprio orologio interno. Gli oscillatori si desincronizzano nel tempo a causa del drift, quindi **non esiste una nozione di tempo fisico globale**. Due eventi su macchine diverse non possono essere ordinati con certezza basandosi solo sui timestamp fisici locali.

Esempio concreto: l'utility `make` di Unix usa i timestamp dei file per capire se ricompilare o meno. Se due macchine in una build distribuita hanno clock leggermente diversi, `make` potrebbe prendere la decisione sbagliata.

Relazione "happens-before" (→)

Lamport introduce la relazione **happens-before** (precede causalmente) per ordinare gli eventi senza bisogno di un clock globale. Le regole sono:

- **Ordine all'interno di un processo**: se a e b sono eventi nello stesso processo e a avviene prima di b, allora $a \rightarrow b$.
- **Comunicazione**: se a è l'evento di invio di un messaggio e b è l'evento di ricezione dello stesso messaggio, allora $a \rightarrow b$ (inviare precede sempre ricevere).
- **Transitività**: se $a \rightarrow b$ e $b \rightarrow c$, allora $a \rightarrow c$.

Se $a \rightarrow b$ significa che a ha *potenzialmente influenzato causalmente* b. Due eventi che non hanno nessuna relazione happens-before sono detti **concorrenti** (o causalmente indipendenti): in questo caso il tempo fisico non aiuta, perché i clock dei due nodi potrebbero essere sfasati.

5.3 Orologi Logici di Lamport

Definizione

Un **orologio logico** è un contatore software che viene incrementato ogni volta che si verifica un evento. Viene usato per assegnare un *timestamp* a ciascun evento, fornendo un ordinamento relativo senza bisogno di un clock fisico sincronizzato.

Regole di aggiornamento

Ogni processo P_i mantiene una variabile contatore CP_i. Le regole sono:

- **Evento interno**: prima di eseguire un evento, P_i incrementa il proprio contatore:

$CP_i := CP_i + 1$

- **Invio di un messaggio**: il processo allega al messaggio il proprio timestamp corrente `tm = CPᵢ`.

- **Ricezione di un messaggio**: il processo `P□` riceve un messaggio con timestamp `tm` e aggiorna il proprio contatore:

`CP□ := max(CP□, tm) + 1`

Il nuovo valore di `CP□` viene usato come timestamp dell'evento di ricezione.

Proprietà e limiti

Queste tre regole garantiscono che se `a → b` allora `timestamp(a) < timestamp(b)`.
Attenzione: il viceversa non è sempre vero. Se `timestamp(a) < timestamp(b)` non possiamo concludere che `a → b` (potrebbero essere concorrenti).

Questo significa che gli orologi logici di Lamport definiscono solo un **ordinamento parziale** degli eventi: eventi su nodi diversi potrebbero avere lo stesso timestamp o un ordinamento ambiguo.

5.4 Ordinamento Totale

Dal parziale al totale

Per molte applicazioni (es. transazioni distribuite) è necessario un **ordinamento totale**: tutti i processi devono concordare sullo stesso ordine di tutti gli eventi, anche tra quelli concorrenti.

La soluzione di Lamport è usare un **timestamp globale** composto da una coppia `(CPᵢ(a), i)`:

- Prima si confronta `CPᵢ(a)`: l'evento con timestamp minore è avvenuto "prima".
- In caso di parità, si usa l'identificatore del processo `i`: il processo con indice minore ha la precedenza.

In questo modo si ottiene un ordinamento totale deterministico e concordato da tutti i processi.

> Nota: l'ordinamento totale **non implica** l'ordinamento causale. Due eventi concorrenti `a` e `b` vengono ordinati in modo arbitrario ma consistente, non necessariamente in base alla causalità. Per le transazioni questo va bene, perché sono per definizione indipendenti l'una dall'altra.

5.5 Vector Clock – Ordinamento Causale

Il problema degli orologi logici di Lamport

Gli orologi di Lamport non permettono di determinare se due eventi sono causalmente correlati o semplicemente concorrenti. Se `timestamp(a) < timestamp(b)` non so se `a → b` oppure se `a` e `b` sono concorrenti.

Per catturare la **causalità** completa tra eventi, si usano i **vector clock** (orologi vettoriali).

Struttura e aggiornamento

Ogni processo P_i mantiene un **vettore** CV_i di dimensione n (dove n è il numero di processi). Ogni entry $CV_i[j]$ rappresenta la "migliore stima" di P_i del clock logico del processo P□.

Le regole di aggiornamento sono:

- **Evento interno in `$P_i$`**: incremento solo la propria entry:

$CV_i[i] := CV_i[i] + 1$

- **Invio di un messaggio da `$P_i$`**: prima incremento, poi allego l'intero vettore al messaggio.

- **Ricezione di un messaggio da `P`□` con vettore `V`**: aggiorno entry per entry prendendo il massimo, poi incremento la mia entry:

Per ogni k: $CV_i[k] := \max(CV_i[k], V[k])$

Poi: $CV_i[i] := CV_i[i] + 1$

Come leggere la causalità

Dati due eventi a in P_i e b in P□ con timestamp vettoriali VA e VB:

Condizione	Significato
$VA < VB$ (tutte le entry di VA ≤ VB e almeno una <)	$a \rightarrow b$ (a precede causalmente b)
$VB < VA$	$b \rightarrow a$ (b precede causalmente a)
Né $VA < VB$ né $VB < VA$	a e b sono **concorrenti** (causalmente indipendenti)

Questa è la proprietà fondamentale: i vector clock permettono di distinguere la causalità dalla semplice concorrenza.

> **Esempio pratico**: nel sistema `make` distribuito, con i vector clock posso capire con certezza se una modifica su una macchina ha preceduto causalmente una modifica su un'altra macchina, indipendentemente dall'orario del clock fisico.

5.6 Sistemi Sincroni e Asincroni

I sistemi distribuiti si classificano lungo un continuum che va dalla piena sincronizzazione alla completa asincronicità.

Sistemi Sincroni

Un sistema è **sincrono** se valgono tutte e tre le seguenti proprietà:

- Esiste un **limite superiore noto** al ritardo di consegna dei messaggi.
- Il **drift rate** degli orologi locali rispetto al clock globale è limitato e noto.
- Esiste un **limite superiore noto** alla velocità di elaborazione di ciascun processore.

In un sistema sincrono è possibile usare i timeout: se non ricevo risposta entro un certo tempo, so con certezza che qualcosa è andato storto.

Sistemi Asincroni

Un sistema è **asincrono** se **non esistono limiti** su nessuna delle tre proprietà precedenti: né sul ritardo dei messaggi, né sul drift degli orologi, né sulla velocità dei processori. In pratica, il sistema è *time-free*.

Implicazione critica: in un sistema asincrono è impossibile distinguere tra:

- Un processo **crashato** (non risponde più).
- Un processo **molto lento** (risponde, ma con ritardo indefinito).
- Un messaggio **perso in rete**.

Questo rende il problema del consensus particolarmente difficile nei sistemi asincroni.

Sistemi Semi-Asincroni (Timed-Asynchronous)

I due modelli presentati sono agli estremi opposti. Nella realtà si lavora spesso con modelli intermedi: si fanno assunzioni probabilistiche sul comportamento del sistema, o si assume che il sistema sia sincrono solo per certi periodi di tempo. Questo porta ai **sistemi timed-asynchronous** che vedremo nel Capitolo 7.

5.7 Consensus Distribution

Il problema del Consensus

L'obiettivo del **consensus distribuito** è fare in modo che un gruppo di processi si accordi su un singolo valore, anche in presenza di guasti.

Le condizioni formali che un algoritmo di consensus deve soddisfare sono:

Proprietà	Descrizione
Termination	**Tutti i nodi sani prima o poi devono prendere una decisione.** È l'unica proprietà di *liveness* del gruppo. Non puoi permettere che i processi rimangano in un loop infinito ad aspettare pacchetti fantasma. Il sistema deve progredire. Un algoritmo che non termina è inutile quanto un server spento.
Uniform Integrity	**Nessun nodo può decidere più di una volta.** La decisione è un vincolo irreversibile, un commit definitivo. Un processo non può cambiare idea a partita in corso solo perché ha ricevuto un pacchetto in ritardo. Ti schieri, sigilli lo stato e ti fermi.
Agreement	**Tutti i nodi sani devono decidere lo stesso identico valore.** Questa è *safety* pura. Se il Nodo A decide "0" e il Nodo B decide "1", hai appena spaccato il sistema in due (*split-brain*). Non importa *cosa* decidono, importa che vadano all'altare tutti con la stessa identica idea
Validity	**Il valore deciso deve essere stato proposto da almeno uno dei nodi.** L'algoritmo non ha il diritto di inventarsi una risposta dal nulla. Se i nodi propongono solo "A" o "B", la decisione finale non può essere "C". Altrimenti non è consenso, è un'allucinazione arbitraria del software.

> **Processo corretto**: un processo che non ha subito guasti durante l'esecuzione.

Modello di guasto più semplice

Il modello di guasto più semplice (e più comune per iniziare) è il **crash failure** (o *fail-stop*): un processo guasto smette semplicemente di funzionare e non invia più messaggi. Non si considera il caso in cui un processo menta o si comporti in modo arbitrario (questo sarebbe un *Byzantine fault*, trattato nel Cap. 6).

5.8 Broadcast Affidabile

Dal Consensus al Broadcast

Il broadcast affidabile è uno strumento fondamentale per implementare il consensus. Esistono diverse varianti, ognuna con garanzie diverse.

Best-Effort Broadcast

Il **best-effort broadcast** è la forma più debole ed è un algoritmo di comunicazione fondamentale nei sistemi distribuiti in cui un nodo mittente trasmette un messaggio a tutti gli altri nodi del sistema, ma la rete non fornisce alcuna garanzia in merito alla consegna, alla latenza o all'ordine dei messaggi. Le sue proprietà sono:

- **Validity**: se `p` e `q` sono entrambi corretti, ogni messaggio inviato da `p` viene prima o poi ricevuto da `q`.
- **Uniform Integrity**: nessun messaggio viene ricevuto più di una volta.

Problema: se il mittente crasha prima di completare l'invio, alcuni processi ricevono il messaggio e altri no. Questo viola l'equità.

Reliable Broadcast

Il **reliable broadcast** aggiunge la proprietà di **Agreement**: se un processo corretto riceve un messaggio `m`, allora tutti i processi corretti alla fine riceveranno `m`.

Implementazione: si usa il principio di *message diffusion*. Quando un processo riceve un messaggio per la prima volta, lo **ritrasmette a tutti gli altri** prima di consegnarlo localmente. In questo modo, anche se il mittente originale crasha, il messaggio si propaga comunque.

```
Algoritmo Reliable Broadcast:
  Per ogni messaggio m ricevuto per la prima volta:
    1. Inoltra m a tutti gli altri processi
    2. Consegna m localmente (R-deliver)
```

Esempio reale: operai che lavorano sui binari ferroviari. Il coordinatore deve comunicare a tutti che il treno sta arrivando. Con il reliable broadcast, se il coordinatore cade prima di raggiungere tutti, chi ha già ricevuto il messaggio lo re-trasmette agli altri.

5.9 Varianti di Broadcast Affidabile

Una volta garantito il reliable broadcast, si possono aggiungere garanzie sull'**ordine di consegna**:

Variante	Garanzia d'ordine	Descrizione
FIFO Broadcast	Ordine FIFO per singolo mittente	I messaggi di uno stesso mittente vengono consegnati nell'ordine in cui sono stati inviati
Causal Broadcast	Ordine causale	Se `a → b` (a precede causalmente b), allora `a` viene consegnato prima di `b` da tutti i processi
Atomic Broadcast	Ordine totale	Tutti i processi corretti consegnano tutti i messaggi nello stesso ordine (totale)

Relazione tra le varianti

- Il **Total Order** non implica il **FIFO Order** e viceversa.
- Se rispettiamo sia il Total Order che il FIFO Order si ottiene un **FIFO Total Order**.
- L'**Atomic Broadcast** è equivalente al problema del Consensus:
- Se un processo vuole proporre un valore, lo fa tramite broadcast.
- La decisione finale è determinata dall'ordine in cui il messaggio viene consegnato dall'Atomic Broadcast.

Il Teorema FLP

Il **teorema FLP** (Fischer, Lynch, Paterson – 1985) è uno dei risultati più importanti della teoria dei sistemi distribuiti:

> **Teorema**: è impossibile risolvere il problema del consensus in un sistema **completamente asincrono** (time-free) anche con un solo processo che può crashare, usando l'Atomic Broadcast.

Intuizione: in un sistema asincrono, un processo non riesce a distinguere se un altro processo è andato in crash o è semplicemente molto lento. Se aspetta, potrebbe aspettare per sempre (violando la proprietà di Termination). Se non aspetta, potrebbe decidere senza aver considerato il contributo del processo lento (violando Agreement o Validity).

Soluzione: il consensus può essere risolto anche in sistemi asincroni usando le altre forme di broadcast (non atomic). Oppure si devono rilassare le assunzioni di asincronismo, come vedremo con i Failure Detector e i sistemi Timed-Asynchronous.

5.10 Revisione: Ordinamenti a Confronto

Ricapitolando i diversi tipi di ordinamento degli eventi in un sistema distribuito:

Ordinamento	Definizione	Strumento
FIFO Order	Per ogni singolo mittente, i messaggi arrivano nell'ordine in cui sono stati inviati	Numero di sequenza per coppia (mittente, ricevitore)
Causal Order	Se $a \rightarrow b$ (relazione happens-before), allora tutti consegnano a prima di b	Vector clock
Total Order	Tutti i processi concordano su un unico ordine globale degli eventi (non necessariamente causale)	Timestamp globale di Lamport
FIFO Total Order	Total Order + FIFO Order	Combinazione dei due

> **Nota importante**: il Causal Order e il Total Order sono concetti ortogonali. Nel Causal Order ci sono spesso molte combinazioni valide (perché gli eventi concorrenti possono essere consegnati in qualsiasi ordine). Aggiungere il Total Order risolve questa ambiguità scegliendo una combinazione specifica e facendola rispettare a tutti i processi.

CAPITOLO 6 – Failure Detector, Consensus Avanzato e Teorema CAP

6.1 Failure Detector

Motivazione

Nel Capitolo 5 abbiamo visto che nei sistemi asincroni è impossibile distinguere un processo crashato da uno lento. I **Failure Detector** (FD) sono un'astrazione che risolve questo problema: si tratta di un **oracolo distribuito** che fornisce indicazioni su quali processi potrebbero essere crashati.

Ogni processo ha accesso a un modulo locale del failure detector, che mantiene una **lista dei processi sospettati** di aver fallito. Caratteristiche fondamentali:

- La lista **non è statica**: un processo può essere *sospettato*, poi scoperto essere ancora vivo e rimosso dai sospettati.
- Due nodi possono avere liste diverse in un dato istante: più il sistema è sincronizzato, più le liste convergeranno.
- Il FD non garantisce di essere sempre corretto, ma fornisce una stima utile per procedere.

Classi di Failure Detector

I FD si classificano in base a due proprietà ortogonali:

Completeness (il FD rileva tutti i processi crashati):

Tipo	Definizione
Strong Completeness	Ogni processo crashato verrà *prima o poi* sospettato da *tutti* i processi corretti
Weak Completeness	Ogni processo crashato verrà *prima o poi* sospettato da *almeno uno* dei processi corretti

> Attenzione: un FD che sospetta *sempre tutti* i processi soddisfa la Strong Completeness ma è chiaramente inutile. Per questo la Completeness va bilanciata con l'Accuracy.

Accuracy (il FD non accusa i processi ancora vivi):

Tipo	Definizione
Strong Accuracy	Nessun processo corretto viene mai sospettato da alcun processo corretto
Weak Accuracy	Esiste *almeno un* processo corretto che non viene mai sospettato
Eventually Strong Accuracy	Esiste un istante di tempo dopo il quale nessun processo corretto viene più sospettato
Eventually Weak Accuracy	Esiste un istante di tempo dopo il quale almeno un processo corretto non viene più sospettato

Classi principali di FD

Combinando Completeness e Accuracy si ottengono le classi principali:

Classe	Completeness	Accuracy
P (Perfect)	Strong	Strong
Q	Weak	Strong
S (Strong)	Strong	Weak
♦S (Eventually S)	Strong	Eventually Weak
W (Weak)	Weak	Weak
♦W (Eventually W)	Weak	Eventually Weak

> È possibile dimostrare che **P** ≅ **Q** (sono equivalenti): un FD di classe Q può essere trasformato in uno di classe P tramite un algoritmo di riduzione. Quindi non ha senso costruire separatamente i due.

Riduzione tra Failure Detector

Teorema fondamentale: è possibile risolvere il consensus con un FD. In particolare esaminiamo come risolverlo con i FD di classe **S** e **♦S**.

In un Failure Detector, l'**Accuracy** è una proprietà "hard" (non sintetizzabile), mentre la **Completeness** è "soft" (può essere potenziata diffondendo i sospetti tra i nodi).

6.2 Algoritmi di Consensus con Failure Detector

Consensus con Failure Detector di Classe S

L'algoritmo si articola in 3 fasi, considerando un sistema con un FD di classe S (Strong Completeness + Weak Accuracy). Questa classe garantisce che almeno un processo corretto non venga mai sospettato, permettendo al sistema di tollerare guasti anche se più della metà dei processi fallisce.

L'algoritmo per la classe S è strutturato in 3 fasi e non pone limiti al numero di fallimenti ($n-1$ crash tollerabili):

- **Fase 1** (Round Asincroni): I processi eseguono $n-1$ round in cui trasmettono i propri valori proposti. In ogni round, un processo attende di ricevere messaggi dagli altri, a meno che il proprio FD locale non li indichi come sospetti.
- **Fase 2** (Accordo sul Vettore): Al termine, i processi corretti rimasti concordano su un vettore V contenente le proposte di tutti o valori nulli ($\perp$) per i processi sospettati.

- **Fase 3** (Decisione): Ogni processo decide il primo valore non nullo presente nella propria copia del vettore V_p.

Caratteristiche:
- Non gestisce processi che entrano dinamicamente nel sistema.
- Funziona anche se più della metà dei processi fallisce (tollera $f < n$ guasti), a patto che l'Accuracy (Weak) sia rigorosamente rispettata fin dall'inizio.

Consensus con Failure Detector di Classe ♦S (Eventually S)

Nel caso reale, mantenere un'accuratezza rigida da subito è difficile. Si utilizza quindi la classe **♦S** (Eventually S), equivalente a **♦W** (Eventually W). Questo crea un confine tra il lato sincrono e quello asincrono del sistema: un periodo asincrono iniziale in cui il FD può sbagliare, seguito da un periodo in cui il FD diventa accurato (sincrono). In questo caso il sistema tollera al piú (n-1)/2 crash failures arrotondato per difetto.

Per risolvere il consensus con ♦S, si usa la tecnica della Rotazione del Coordinatore (**Coordinator Rotation**):

Ogni round r è gestito da un coordinatore p_i (dove $i = (r \mod n) + 1$) ed è diviso in 4 sotto-fasi:

1. **Invio Stime**: Ogni processo invia al coordinatore la propria stima del valore da decidere, corredata da un timestamp.
2. **Proposta del Coordinatore**: Il coordinatore raccoglie la maggioranza delle stime, seleziona quella con il timestamp più alto e la propone a tutti.
3. **Adozione o Sospetto**: I processi ricevono la proposta e inviano un ACK (accettazione) oppure, se il FD locale sospetta il coordinatore, inviano un NACK.
4. **Decisione**: Il coordinatore attende la maggioranza delle risposte ($\lceil (n+1)/2 \rceil$). Se sono tutti ACK, esegue un Reliable Broadcast del valore deciso; altrimenti, si passa al round successivo con un nuovo coordinatore.

6.3 Consensus con Flooding (Flooding-Based Consensus)

Il **flooding-based consensus** è un approccio più diretto, basato sulla semantica *fail-stop*:

Setup

- Un gruppo di processi `P = {p1, p2, ..., pn}`.
- I guasti seguono la semantica fail-stop (un processo guasto si ferma e non invia più nulla).
- Un client contatta un processo per proporre un comando.
- Ogni processo mantiene una **lista dei comandi proposti**.

Algoritmo

1. Il Modello di Sistema

L'algoritmo opera in un contesto specifico:

- Gruppo di Processi: Definito come $P = \{P_1, ..., P_n\}$.
- Semantica Fail-Stop: Si assume che i guasti siano rilevabili in modo affidabile (reliable failure detection).
- Gestione Comandi: Ogni processo mantiene una lista di comandi proposti da eseguire.

2. Funzionamento del Protocollo a Round

L'algoritmo si sviluppa in round iterativi:

- Fase di Multicast: In ogni round r, ogni processo P_j invia il proprio set di comandi conosciuti C_i^r a tutti gli altri membri.
- Fase di Merge: Alla fine del round, ogni processo P_i unisce i set ricevuti in un nuovo insieme aggiornato C_i^{r+1}.
- Selezione Deterministica: Il comando successivo viene scelto applicando una funzione deterministica condivisa a livello globale (es. select) sulla lista risultante.
- Convergenza: Il sistema funziona correttamente se e solo se tutti i processi corretti terminano il round con la stessa lista di comandi.

Problema principale

Se un processo fallisce durante un round, gli altri non sanno se hanno tutti ricevuto le stesse informazioni. Nel caso in cui uno dei processi abbia ricevuto tutte le informazioni è in grado di passare al turno successivo facendo il broadcast della sua lista di comandi aggiornata. Una volta che gli altri processi ricevono anche loro i comandi mancanti possono continuare l'algoritmo.

> **Nota**: un algoritmo più sofisticato e molto utilizzato in pratica è **Paxos**, che estende il flooding-based consensus con garanzie più robuste. Paxos non è trattato nel corso ma è presente nel libro di testo.

6.4 Consensus Bizantino

Il problema dei generali bizantini

Il nome deriva da un problema storico-militare: un esercito bizantino circonda una città nemica. I generali di ogni divisione devono accordarsi sull'attaccare o ritirarsi, usando solo messaggeri. Il problema è che **alcuni generali potrebbero essere traditori** e inviare messaggi contraddittori per sabotare l'accordo.

Nei sistemi distribuiti, un **Byzantine fault** (guasto Bizantino) è un guasto in cui un processo non smette di funzionare, ma si comporta in modo **arbitrario o malevolo**: può inviare messaggi falsi, modificare i dati, rispondere in modo diverso a processi diversi.

I guasti crash-stop sono un caso speciale di Byzantine fault (un processo che tace è tecnicamente "arbitrario" nel senso che non contribuisce).

Scenari critici

Due situazioni problematiche:

- **Il comandante supremo è un traditore**: invia ordini diversi alle varie divisioni. Anche se tutti i generali onesti si scambiano i messaggi, non riescono a capire quale fosse l'ordine originale corretto.

- **Un generale intermedio è un traditore**: riceve un ordine corretto ma lo trasmette in modo errato agli altri usando il flooding.

Quanti processi servono?

Per tollerare `f` processi Bizantini (traditori) in un sistema con `n` processi totali, è necessario che:

$$\mathbf{n \geq 3f + 1}$$

Intuizione: con 3 processi di cui 1 traditore (f=1), non si può raggiungere il consensus (la maggioranza richiede almeno 2 processi corretti su 3, ma il traditore può sempre creare ambiguità). Con 4 processi di cui 1 traditore si può.

Soluzione standard: si sceglie un **valore di default** (es. "ritirata"). Se la maggioranza non viene raggiunta, tutti scelgono il valore di default.

Firma digitale come soluzione

Inizialmente il problema Bizantino sembrava puramente teorico. Con l'avvento degli attacchi informatici reali, è diventato improvvisamente rilevante. La soluzione moderna è usare la **firma digitale**:

- Ogni messaggio viene firmato con la chiave privata del mittente.
- Il ricevitore può verificare l'autenticità con la chiave pubblica.
- Se il comandante è un traditore, la firma rivela la contraddizione: ha firmato due messaggi diversi, il che è rilevabile.

Con la firma digitale (e l'autenticazione dei messaggi), il requisito minimo di nodi corretti per raggiungere il consenso si riduce (spesso a $n \geq f + 1$ o $n \geq 2f + 1$) rispetto al requisito $n \geq 3f + 1$ necessario in assenza di autenticazione.

6.5 Teorema CAP

Definizione

Il **teorema CAP** (Brewer, 2000) afferma che in un sistema distribuito è impossibile garantire simultaneamente tutte e tre le seguenti proprietà:

Proprietà	Definizione
Consistency (C)	Tutti i nodi vedono gli stessi dati nello stesso momento (consistenza forte)
Availability (A)	Ogni richiesta riceve sempre una risposta (successo o fallimento), senza timeout
Partition Tolerance (P)	Il sistema continua a funzionare anche se una parte della rete smette di comunicare con un'altra

Comprensione intuitiva

Consideriamo due processi `A` e `B` che non riescono più a comunicare (partizione di rete):

- Se vogliamo mantenere la **Consistency**: dobbiamo bloccare le write su `A` finché non si risincronizza con `B`. Ma così `A` non è più **Available**.
- Se vogliamo mantenere la **Availability**: `A` può ancora accettare write, ma i dati di `A` e `B` divergono, violando la **Consistency**.
- Se vogliamo ignorare la partizione (assumere che la rete non si rompa): stiamo violando la **Partition Tolerance**.

Conclusione: CAP si riduce a un trade-off tra *liveness* (disponibilità) e *safety* (consistenza).

Scelte progettuali reali

In pratica, la Partition Tolerance è quasi sempre necessaria (le reti si rompono). Quindi la scelta reale è tra:

- **CP (Consistency + Partition Tolerance)**: il sistema preferisce essere corretto piuttosto che disponibile. In caso di partizione, rifiuta alcune richieste. Es: sistemi bancari, database relazionali distribuiti.
- **AP (Availability + Partition Tolerance)**: il sistema preferisce rispondere sempre, accettando che i dati possano essere temporaneamente inconsistenti. Es: DNS, sistemi NoSQL come Cassandra o DynamoDB.

Eventual Consistency

Come si gestisce l'AP? Si accetta una **eventual consistency** (consistenza eventuale): i dati divergono temporaneamente durante una partizione, ma quando la rete si ripristina il sistema converge verso uno stato consistente tramite meccanismi di *cold recovery* (riconciliazione).

6.6 Distributed Commit – Two-Phase Commit (2PC)

Il problema del commit distribuito

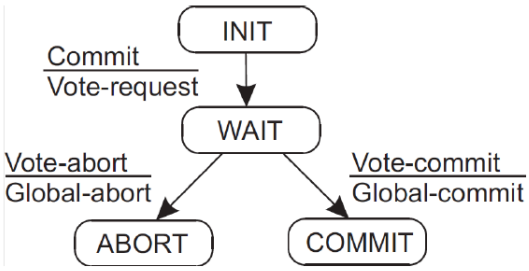
In un sistema distribuito, un'operazione potrebbe dover essere eseguita atomicamente su più nodi. Il **distributed commit** garantisce che tutti i nodi eseguano l'operazione oppure nessuno lo faccia (atomicità distribuita).

Protocollo Two-Phase Commit (2PC)

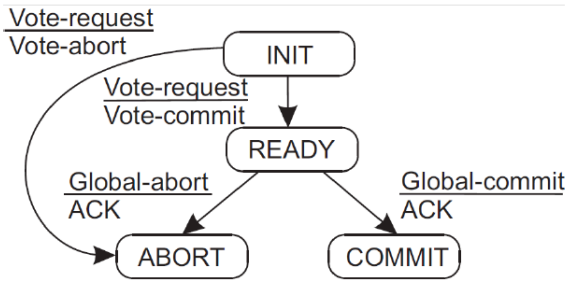
Il 2PC assume l'assenza di guasti ma gestisce nodi che non riescono a fare commit. Si articola in **due fasi**:

Fase 1 – Voting (Preparazione):
- Il **coordinatore** invia un messaggio `PREPARE` a tutti i partecipanti.
- Ogni partecipante risponde `YES` (può fare commit) o `NO` (non può).
- I partecipanti che rispondono `YES` entrano in stato `READY` e non possono più fare commit autonomamente: devono aspettare l'ordine del coordinatore.
- Se ricevono un `ABORT` dal coordinatore, tutti abortiscono.

Fase 2 – Commit/Abort:
- Se **tutti** i partecipanti hanno votato `YES`, il coordinatore invia `GLOBAL COMMIT`.
- Se **almeno uno** ha votato `NO`, il coordinatore invia `GLOBAL ABORT`.
- I partecipanti eseguono il commit o l'abort e inviano un `ACK`.



Coordinator



Participant

Gestione dei guasti

In un sistema reale i guasti avvengono. Alcuni scenari:

Timeout del partecipante in stato INIT (non ha ricevuto il PREPARE):

- Il partecipante manda autonomamente un `ABORT` dopo il timeout. Sicura perché non ha ancora votato.

Timeout del coordinatore in stato WAIT (non ha ricevuto tutti i voti):

- Il coordinatore invia un `GLOBAL ABORT`. I partecipanti che erano in READY vengono informati.

Partecipante bloccato in stato READY (ha votato YES ma non riceve la decisione):

- È il caso più critico: il partecipante *non può* decidere autonomamente (non sa se gli altri hanno votato YES). Deve contattare altri partecipanti o aspettare il recupero del coordinatore.

Recovery dopo crash

Quando un partecipante crasha e si riavvia, deve riportarsi allo stato coerente:

Stato al crash	**Azione al recovery**
INIT	Nessun problema, non era ancora nel protocollo
READY	Deve contattare il coordinatore per sapere la decisione globale
ABORT	Rimuove il workspace (annulla le modifiche locali)
COMMIT	Copia il workspace nel database (applica le modifiche)

Crash del coordinatore: se il coordinatore crasha in stato WAIT, un partecipante può re-inviare il VOTE-REQUEST dopo un timeout. Questo però funziona solo in sistemi **semi-asincroni** (con timeout definiti): in un sistema puramente asincrono il timeout non ha senso.

6.7 Recovery e Checkpointing

Recovery = ritorno del sistema in uno stato "error-free".

Tipi di recovery

Tipo	Descrizione
Forward Recovery	Il sistema si riprende dal guasto e continua l'esecuzione da un *nuovo* stato (correzione in avanti)
Backward Recovery	Il sistema torna a uno stato precedente al guasto (*rollback*) e riprende da lì

In pratica si usa quasi sempre il **Backward Recovery**, definendo *recovery points* (checkpoint) periodici.

Coordinated Checkpointing

Tutti i processi prendono un checkpoint in modo coordinato:

- Il coordinatore invia un multicast di "checkpoint now".
- I partecipanti prendono il checkpoint e inviano un `ACK`.
- Il coordinatore aspetta tutti gli ACK e poi invia "checkpoint done".

Vincolo critico: nessun processo deve compiere operazioni tra il checkpoint e il "checkpoint done". Questo può essere irrealizzabile in sistemi ad alto throughput.

Uncoordinated Checkpointing

Ogni processo prende checkpoint autonomamente. Il rischio è il **cascading rollback**: se al momento del crash i checkpoint non sono consistenti tra loro, il recovery richiede di tornare indietro a stati sempre più vecchi, propagando il rollback a cascata.

Notazione:

- $CP_i(m)$: il checkpoint m-esimo del processo P_i
- $INT_i(m)$: l'intervallo tra $CP_i(m)$ e $CP_i(m+1)$
- Le dipendenze tra intervalli vengono registrate: se P_i invia un messaggio in $INT_i(m)$ e P_j lo riceve in $INT_j(n)$, si registra la dipendenza $INT_i(m) \rightarrow INT_j(n)$.
- La dipendenza $INT_i(m) \rightarrow INT_j(n)$ viene salvata nella memoria quando si effettua il checkpoint $CP_j(n)$.

Osservazione:
Se il processo P_i viene annullato e riportato a $CP_i(m-1)$, P_j deve essere annullato e riportato a $CP_j(n-1)$. Se $CP_j(n-1)$ non porta a ulteriori incongruenze, va bene. Altrimenti, si procede con un'ulteriore annullamento (**cascading rollback**).

Message Logging

Per ridurre il costo del rollback si usa il **message logging**: siccome ogni processo, in ogni stato, compie operazioni deterministiche, registrando la sequenza degli stati è possibile **riprodurre** il comportamento del processo anche dopo un crash.

Notazione:

- $DEP(m)$: insieme dei processi a cui m è stato consegnato (inclusi quelli che dipendono causalmente da m).
- $COPY(m)$: processi che hanno una copia di m non ancora salvata stabilmente.
- $FAIL$: insieme dei processi che hanno fallito.

Caratterizzazione:
Un processo si dice orfano quando sopravvive al crash di un altro processo, ma il cui stato è incoerente con quello del processo crashato dopo il suo ripristino. Quindi, per un processo Q:

$$Q \text{ è orfano} \Leftrightarrow \exists m: Q \in DEP(m) \text{ e } COPY(m) \subseteq FAIL$$

Per evitare processi orfani, dobbiamo assicurarci che se ogni processo in $COPY(m)$ si arresta in modo anomalo, non rimanga alcun processo sopravvissuto in $DEP(m)$.

Schemi di logging:

- **Pessimistic logging**: per ogni messaggio non ancora stabile, *al massimo un processo* dipende da esso ($|DEP(m)| \leq 1$). Sicuro ma costoso.
- **Optimistic logging**: si accetta che più processi dipendano da un messaggio non stabile, ma si garantisce che se tutti i processi che hanno una copia del messaggio falliscono, anche tutti i dipendenti finiranno per fallire ($COPY(m) \subseteq FAIL \rightarrow DEP(m) \subseteq FAIL$). Più efficiente ma più complesso.

CAPITOLO 7 – Sistemi Timed-Asynchronous e Consensus per Blockchain

7.1 Sistemi Timed-Asynchronous

Motivazione

Il modello puramente asincrono è troppo pessimistico per il mondo reale: un utente che naviga su internet sa benissimo che se una pagina non carica dopo 30 secondi c'è un problema, e chiude o ricarica. Gli esseri umani ragionano naturalmente con il tempo; i sistemi reali hanno orologi locali, anche se imperfetti.

Il modello **timed-asynchronous** è una via di mezzo tra il sincrono e l'asincrono: introduce clock locali con drift limitato e timeout definiti, senza però richiedere sincronizzazione globale.

Assunzioni del modello

Il sistema timed-asynchronous fa le seguenti assunzioni:

- **Tutti i servizi sono temporizzati**: ogni operazione può definire un timeout.
- **La comunicazione avviene tramite datagram** (non reliable): i messaggi possono perdersi ma non vengono corrotti.
- **I processi possono subire crash e timing failure** (nessun guasto Bizantino).
- **I processi hanno accesso a clock locali con drift limitato**: in ogni istante il drift è nell'intervallo $[-\rho, +\rho]$ per una costante ρ nota, ma non deve essere costante, solo entro quel limite.
- **Non esiste un limite superiore al tasso di guasto dei processi**: i crash sono eventi normali e attesi.

Datagram Service

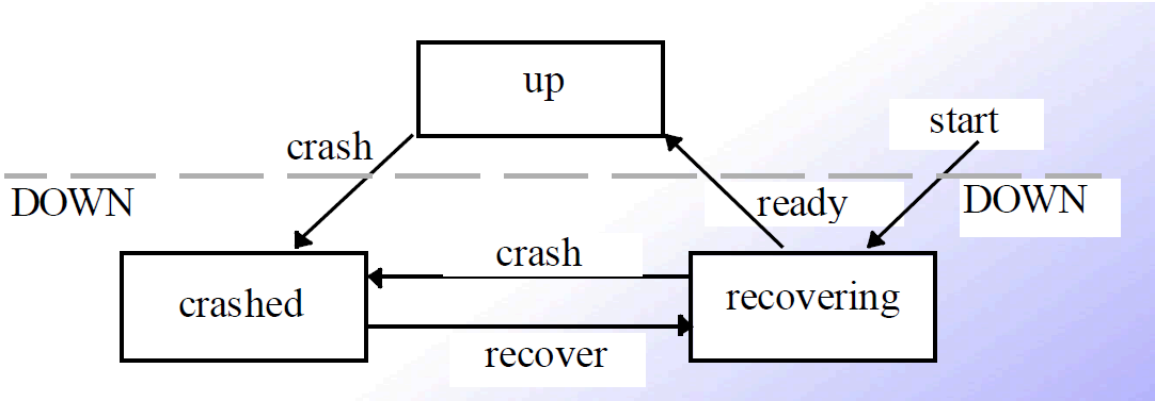
Le proprietà del servizio datagram in questo modello sono:

- I messaggi sono identificati univocamente (unicast e broadcast).
- **Non esiste un limite superiore al ritardo** di trasmissione.
- Esiste un **timeout** per la trasmissione: se il messaggio non arriva entro il timeout, si considera perso.
- Il tempo di trasmissione è proporzionale alla dimensione del messaggio.
- Si considerano solo guasti di **omissione e temporizzazione** (i messaggi corrotti sono ignorati perché improbabili).

Stati dei processi

Un processo in un sistema timed-asynchronous si trova in uno di tre stati:

Stato	Descrizione
Up	Il processo funziona correttamente
Crashed	Il processo ha smesso di funzionare
Recovering	Il processo si sta riprendendo da un crash



> Nota: un crash dell'orologio hardware causa quasi sempre il crash del processo, ma non vale il viceversa (il processo può crashare per altri motivi lasciando l'orologio intatto).

7.2 Predicati di Stabilità

Nei sistemi timed-asynchronous si usano **predicati di stabilità** per verificare condizioni favorevoli del sistema – condizioni in cui il sistema si comporta come se fosse sincrono.

Definizioni

Processo timely: Un processo si dice timely (in italiano tempestivo o puntuale) quando avviene nel momento opportuno, adatto o previsto. Ovvero, il processo avviene né troppo presto né troppo tardi, ma esattamente quando serve.
In termini di sistemi operativi, un processo non è *timely* se subisce lunghi stop dovuti alla Garbage Collection, se va in *thrashing* sulla memoria swap, o se lo *scheduler* della CPU lo penalizza privandolo di cicli macchina. Non significa che debba essere "istantaneo", ma che le sue performance sono **prevedibili e limitate superiormente**.

Connessione tra due processi: due processi p e q sono *connessi* nell'intervallo $[s, t]$ se sono entrambi *timely* in quell'intervallo e ogni messaggio scambiato tra loro subisce un ritardo massimo di δ (il timeout di andata, one-way timeout delay).

Maggioranza stabile: un insieme `S` di processi è una *majority-stable* se ogni coppia di processi in `S` è connessa in un certo intervallo `[s, t]`.
Questo è il concetto chiave. Dal punto di vista della teoria dei grafi, un insieme S di processi è una maggioranza stabile se nell'intervallo, forma un **clique (un sottografo completo) perfettamente sincrono**.

Sistema majority-stable: un sistema è majority-stable in un intervallo se esiste una maggioranza stabile in quell'intervallo.

Processo majority-stable: un processo `p` è majority-stable in un intervallo se è connesso a una maggioranza stabile in quell'intervallo.

Assunzione di Stabilità Eventuale

Un sistema è **always eventually majority-stable** se:

- Dopo ogni periodo di instabilità, il sistema diventa prima o poi majority-stable per almeno `δ` unità di tempo.
- Ogni processo diventa prima o poi majority-stable per almeno `δ` unità di tempo (oppure subisce definitivamente un crash).

Condizioni di Terminazione

In un sistema always eventually majority-stable: se un processo `p` è majority-stable nell'intervallo `[t, t+E]`, allora un'operazione avviata all'istante `t` deve terminare entro `t + E`.

Progress Assumptions

Le assunzioni di progresso permettono di risolvere problemi come il consensus:

> Infinitamente spesso esisterà un intervallo di tempo in cui la maggioranza dei processi sarà stabile.

Quando il sistema è stabile si comporta esattamente come un sistema sincrono, permettendo di completare le computazioni in tempo finito.

7.3 Consensus con Leadership Rotante (Rotating Leadership)

Questo algoritmo risolve il consensus in un sistema timed-asynchronous con le seguenti assunzioni:

- In ogni istante esiste **al più un leader**.
- Solo un processo **majority-stable** può essere eletto leader.
- Un processo rimane leader per un **tempo limitato** (LD unità di clock).
- Il processo sa di essere leader (ma non è richiesto che gli altri lo sappiano).
- I clock dei processi sono **sincronizzati** entro una costante (deviazione massima nota).

Algoritmo di elezione del leader

- All'inizio di ogni *time-slot*, ogni processo è candidato all'elezione come leader.
- Ogni processo è associato a una **priorità**: il protocollo di elezione garantisce che solo il processo con massima priorità venga eletto.
- Per diventare leader, un processo deve ricevere **una maggioranza di risposte** alle proprie candidature **entro il tempo stabilito**.
- Dopo aver inviato la candidatura, ogni processo aspetta un certo periodo per ricevere le candidature degli altri prima di rispondere a quella con priorità più alta.
- Dopo essere diventato leader, il processo rimane tale per **LD unità di clock**, poi decade.

> Il meccanismo funziona grazie al drift limitato: si può costruire un *real-time envelope* (una finestra temporale garantita) entro cui l'elezione avviene correttamente anche con clock non perfettamente allineati.

Azioni del leader eletto

Quando un processo diventa leader:

- Fa un **broadcast** per sapere se gli altri processi conoscono già una decisione o una proposta precedente.
- Aspetta 2δ unità di clock per ricevere le risposte. Poi:
- **Se apprende che un processo ha già deciso per `w`**: decide anch'esso w e lo comunica a tutti.
- **Se nessuno conosce una proposta precedente**: propone il proprio valore iniziale.
- **Se conosce una proposta precedente**: propone quel valore (non il proprio).
- **Se non riceve un numero sufficiente di risposte**: non prende nessuna azione (il round fallisce).

Ricezione di una proposta da parte degli altri processi

Quando un processo p riceve una proposta dal leader:

- Controlla che arrivi entro il limite temporale massimo Λ (errore massimo dell'algoritmo di sincronizzazione dei clock).
- Verifica che abbia priorità più alta dell'ultima proposta vista.
- Se entrambe le condizioni sono soddisfatte: salva il valore e la priorità della proposta, risponde con un $\texttt{ACK}$.

Proprietà di correttezza

- Quando il leader decide un valore `v`, una maggioranza di processi conosce già la proposta `v`.
- Un processo che si riavvia (*restart*) deve re-inizializzare il proprio stato prima di tornare allo stato `UP`.
- In questo modo l'invariante è rispettato anche quando processi che conoscevano la proposta vengono sostituiti da altri dopo un crash.

7.4 Blockchain e Consensus Distribuito

Storia e contesto

La **blockchain** è la tecnologia che sta alla base delle criptovalute, in particolare di Bitcoin, in quanto è un registro pubblico, e distribuito, che registra le transazioni in bitcoin. Il termine indica una catena di blocchi crittograficamente collegati tra loro. Il suo punto di forza è essere **decentralizzata** e **peer-to-peer**: non esiste un'autorità centrale.

Cronologia:

- **2008**: viene registrato bitcoin.org e pubblicato il paper di *Satoshi Nakamoto* (pseudonimo, identità ancora sconosciuta). Introduce il software per il mining di Bitcoin.
- Esistevano tecnologie simili precedenti, ma erano meno usabili e meno ben progettate.

Evoluzione della blockchain:

Generazione	Uso principale
Blockchain 1.0	Criptovalute (Bitcoin, Litecoin) La prima generazione nasce esclusivamente per decentralizzare la moneta e i sistemi di pagamento senza intermediari. Dal punto di vista sistemistico, **la blockchain funziona come un semplice libro mastro contabile** (*ledger*) che registra il passaggio di asset nativi da un indirizzo all'altro. Lo stato del sistema è statico: traccia solo chi possiede cosa (tramite il modello UTXO). Non c'è spazio per la logica di business o per l'esecuzione di codice complesso; lo scripting interno è minimale e privo di cicli per garantire la massima sicurezza e prevedibilità della rete contro attacchi Denial of Service.

Blockchain 2.0	Smart Contracts (Ethereum) Con la seconda generazione, il registro transazionale evolve in un computer globale distribuito. Viene introdotto il concetto di *Smart Contract*: porzioni di codice immutabili ed eseguibili autonomamente dai nodi della rete al verificarsi di determinate condizioni. **Lo stato del sistema diventa dinamico e persistente**, gestito tramite un modello ad account che memorizza non solo i saldi, ma anche variabili e dati complessi. Per gestire l'esecuzione di linguaggi Turing-completi senza rischiare il blocco dei nodi causato da loop infiniti, viene introdotto un sistema di computazione a consumo (il *Gas*).
Blockchain 3.0	Applicazioni distribuite (DApp), identità digitale, supply chain La **terza generazione** si focalizza sul superamento dei vincoli strutturali di throughput e latenza, mirando a prestazioni di **livello industriale**. L'architettura evolve dalla linearità classica verso strutture dati parallele (come lo *sharding* o i grafi orientati *DAG*) e protocolli di consenso avanzati (*Proof of Stake* e varianti *BFT*) per minimizzare i tempi di finalità. L'obiettivo primario è la creazione di un ecosistema **interoperabile** capace di connettere reti eterogenee e supportare l'esecuzione di *DApp* (Decentralized Applications) complesse su vasta scala.

Come funziona in generale

- Ogni **10 minuti** viene aggiunto un nuovo blocco alla catena.
- Ogni blocco contiene tutte le transazioni avvenute negli ultimi 10 minuti.
- Il blocco viene **trasmesso** (broadcast) alla rete e la rete deve accordarsi sulla sua validità.
- L'autenticazione avviene tramite **chiave privata**: se perdi la chiave allora perdi l'accesso alle tue risorse per sempre. Non esiste recupero.

Tipi di Consensus in Blockchain

Tipo	Caratteristiche
Permissionless consensus	Tutti i nodi hanno gli stessi diritti e le stesse possibilità; chiunque può partecipare senza chiedere permesso. Es: Bitcoin.
Permissioned consensus	I nodi possono avere diritti diversi; è necessaria un'autorizzazione per partecipare. Es: blockchain aziendali.

Terminologia:

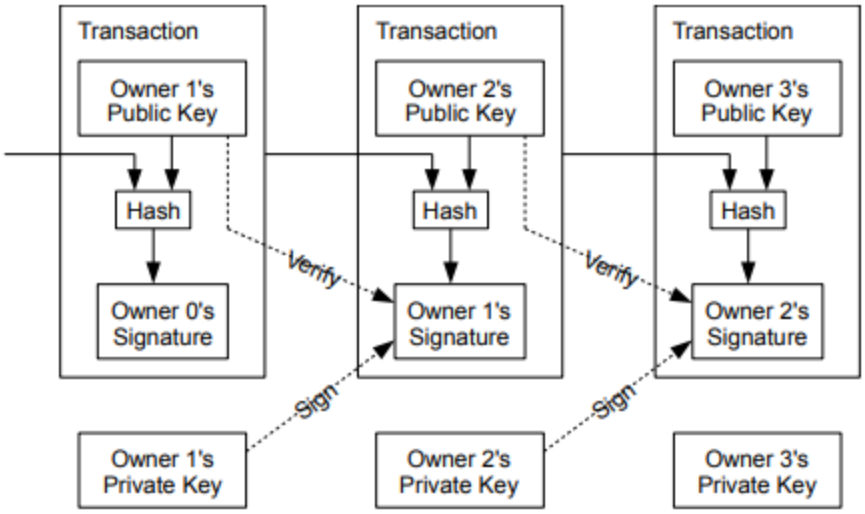
- **Mining**: il processo di raggiungimento dell'accordo (consensus).
- **Blocco**: la struttura dati base della blockchain. È composto da Header e Corpo:
  - L'**Header** contiene: l'hash del blocco precedente, il timestamp di creazione del blocco, il nonce (numero pseudo-casuale utilizzato dall'algoritmo di consenso), la merkle root
  - Il **Corpo** contiene l'insieme delle transazioni validate
- **Miner**: chi partecipa all'algoritmo di consensus.

7.5 Proof of Work (PoW)

Meccanismo base

Il **Proof of Work** è il primo algoritmo di consensus implementato in Bitcoin. È **permissionless**.

Ogni transazione è firmata digitalmente: il proprietario corrente trasferisce la moneta al successivo firmando digitalmente un hash della transazione precedente concatenato con la chiave pubblica del nuovo proprietario. Chi riceve la moneta può verificare l'intera catena di firme.



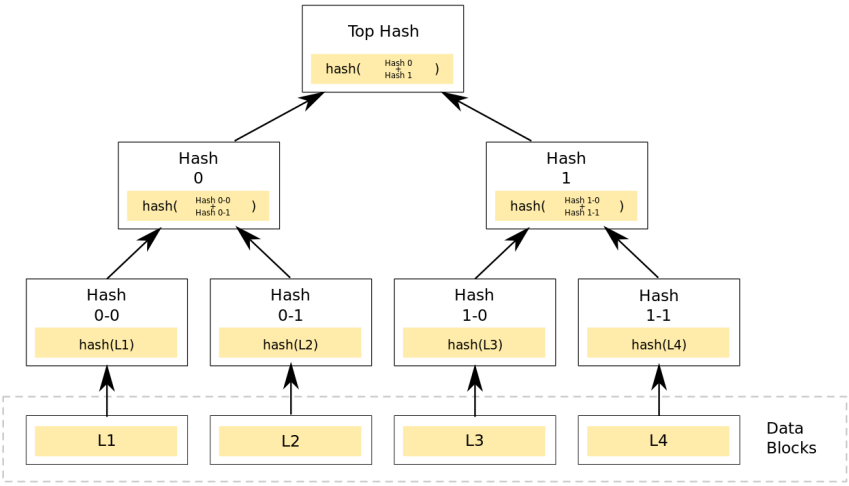
Problema del double spending: chi compra una moneta non sa se il venditore l'ha già spesa altrove.

Soluzione: ogni transazione deve essere annunciata pubblicamente. In questo modo tutti concordano su una singola catena valida, rendendo impossibile spendere due volte la stessa moneta.

Timestamp server e Merkle Tree

Un **timestamp server** funziona come un giornale: contiene le informazioni sulla catena e indica qual è la catena corretta in un dato momento.

Le transazioni sono organizzate in un **Merkle Tree** (Hash Tree), una struttura dati ad albero dove ogni foglia è l'hash di una transazione e ogni nodo interno è l'hash dei propri figli. Solo la radice dell'albero, **Merkle root**, è inclusa nell'hash del blocco. Quindi la Merkle root è l'hash crittografico finale che riassume tutti gli hash delle transazioni all'interno di un blocco blockchain, funzionando come un'impronta digitale sicura. Questo permette di **compattare** la blockchain: una volta che una transazione è sepolta sotto abbastanza blocchi, i rami più vecchi del Merkle Tree possono essere eliminati senza rompere la struttura crittografica.



Come si ottiene una moneta (Mining)

Per ottenere una moneta (ricompensa), un miner deve trovare un valore (*nonce*) tale che, quando aggiunto al blocco e hashato, l'hash risultante inizi con un certo numero di zeri. Questa è una **Proof of Work**: dimostra che il miner ha effettivamente speso potenza computazionale.

Algoritmo del Proof of Work

- Le nuove transazioni vengono trasmesse in broadcast a tutti i nodi.
- Ogni nodo raccoglie le nuove transazioni in un blocco.
- Ogni nodo lavora per trovare un *proof-of-work* difficile per il proprio blocco.
- Quando un nodo trova il proof-of-work, trasmette il blocco a tutti.
- I nodi accettano il blocco solo se tutte le transazioni in esso sono valide e non già spese.
- I nodi esprimono l'accettazione lavorando sul blocco successivo, usando l'hash del blocco accettato come *previous hash*.

Conflitti: catena più lunga vince

Se due nodi trovano simultaneamente un proof-of-work per blocchi diversi (fork), si crea una biforcazione temporanea. La regola è: **vince la catena più lunga**. I nodi tengono entrambe le versioni finché una non diventa più lunga, poi scartano l'altra.

Problemi del PoW

- **Throughput bassissimo**: solo 1 blocco ogni 10 minuti, con soli ~1MB di dati → circa 7 transazioni al secondo (contro le migliaia di Visa).

- **Consumo energetico enorme**: per minare un singolo blocco serve più energia di una casa americana in un anno.
- **Attacco del 51%**: se un singolo nodo controlla più del 51% della potenza di hash totale, può riscrivere la blockchain. In teoria possibile ma praticamente impossibile su Bitcoin.

7.6 Proof of Stake (PoS)

Il **Proof of Stake** è un'alternativa al PoW che non richiede mining ad alta potenza computazionale.

- Ogni **10 secondi** (molto più frequente del PoW) viene selezionato casualmente un validatore.
- La probabilità di essere selezionato è proporzionale alla quantità di criptovaluta che il validatore ha messo in *stake* (bloccato come garanzia).
- Il validatore selezionato crea il blocco successivo.
- I blocchi sono collegati al proprio padre
- Normalmente la catena converge in una singola catena.

Vantaggi rispetto a PoW:
- Nessun consumo energetico massiccio.
- Più veloce.

Svantaggi:
- Chi ha più ricchezza ha più potere (problema di centralizzazione).
- Attacchi "nothing at stake": un validatore malevolo potrebbe votare per più fork contemporaneamente senza alcun costo, introduzione di una **penalità** (slashing) che viene applicata al validatore in caso di comportamento malevolo.

7.7 Altre varianti: Proof of X

Esistono molte varianti degli algoritmi di consensus per blockchain, tutte basate su una *prova* di qualcosa:

Algoritmo	**Prova richiesta**
Proof of Deposit	Deposito di una somma come garanzia
Proof of Coin Age	Monete tenute per un certo periodo
Proof of Identity	Verifica dell'identità reale del partecipante
Proof of Elapsed Time (PoET)	Dimostrazione di aver atteso un tempo casuale. Non serve alta potenza computazionale. Il problema è che la TEE

	(Trusted Execution Environment) è hardware: se qualcuno riesce a manomettere la TEE può accedere a molti più contratti del dovuto.

7.8 PBFT – Practical Byzantine Fault Tolerance

Il **PBFT** è il primo algoritmo basato su BFT (Byzantine Fault Tolerance) usato in blockchain.

Caratteristiche fondamentali

- Si basa su **repliche deterministiche** di un server: se non falliscono, producono lo stesso risultato per lo stesso input.
- Con `f` repliche guaste, l'algoritmo funziona con **n = 3f + 1** nodi totali.
- È un algoritmo **permissioned** (non permissionless come PoW).

Concetti chiave

- I nodi **client** inviano richieste di transazioni.
- Il protocollo usa l'approccio **primary-backup**:
  - I nodi **replica** sono responsabili della finalizzazione dei blocchi.
  - Si muovono in configurazioni diverse chiamate **view** (cambiate a ogni round).
  - Ogni view ha **una replica primaria** (che crea e finalizza blocchi) e **repliche di backup** (che finalizzano blocchi).
  - La replica primaria sceglie l'ordine di esecuzione delle richieste dei client.
  - La replica primaria assegna un numero alla richiesta e la invia ai backup.
  - La replica primaria potrebbe fallire => i backup utilizzano un timeout e ordinano un cambio di view.

Il Ciclo di Vita di una Transazione (Fasi del Protocollo)

Il flusso logico per confermare e sigillare una transazione richiede cinque step. Non ci si fida di nessuno finché la matematica non lo impone.

1. **Request**: Il nodo client invia una richiesta di transazione al nodo Primary.
2. **Pre-prepare** (Dittatura iniziale): Il Primary raccoglie la richiesta, le assegna un numero di sequenza univoco (decide l'ordine) e la trasmette in broadcast (relazione $1 \rightarrow n$) ai Backup tramite un messaggio Pre-prepare.
3. **Prepare** (Consenso Locale): I Backup ricevono la Pre-prepare, la validano e inviano in broadcast (relazione $n \rightarrow n$) un messaggio di Prepare a tutti gli altri nodi.
   - Stato Interno: Quando un nodo colleziona il messaggio Pre-prepare originale in aggiunta a $2f$ messaggi di Prepare concordanti, ottiene un Prepared

Certificate. In questo momento, la maggioranza onesta concorda sull'ordine delle transazioni, ma l'esecuzione non è ancora globale.

4. **Commit** (Consenso Globale): Appena un nodo ottiene il Prepared Certificate, spara in broadcast un messaggio di Commit ($n \to n$).
    - Stato Interno: Quando un nodo colleziona $2f + 1$ messaggi di Commit concordanti, ottiene il Committed Certificate. La transazione ora è matematicamente irreversibile. Il nodo la esegue nel suo database locale.
5. **Reply** (La garanzia per il Client): Ogni nodo, appena eseguita la transazione, invia il risultato (Reply) direttamente al client.
    - Il client, che ignora totalmente le dinamiche interne della rete, non si fida della singola risposta. Rimane in ascolto finché non riceve $f + 1$ risposte identiche da repliche diverse. Per la regola del quorum, è matematicamente certo che almeno uno di quei nodi sia onesto, sigillando la validità della richiesta.

Confronto tra algoritmi di consensus

Algoritmo	Tipo	Tolleranza ai guasti	Throughput	Uso energetico
PoW	Permissionless	Crash + Sybil	Basso	Altissimo
PoS	Permissionless	Crash	Medio	Basso
PBFT	Permissioned	Byzantine (f < n/3)	Alto	Basso

7.9 Smart Contract

Uno **smart contract** è un contratto *auto-eseguibile*: i termini dell'accordo tra le parti sono scritti direttamente in codice, che vive nella blockchain. Non richiede intermediari (notai, avvocati, banche).

Come funziona: il codice viene caricato sulla blockchain in modo corretto e viene eseguito automaticamente al verificarsi delle condizioni definite. È immutabile e trasparente.

Esempi di applicazione:
- **NFT** (Non-Fungible Token): token unici che certificano la proprietà di un asset digitale (arte, musica, oggetti di gioco).
- **DeFi** (Decentralized Finance): applicazioni finanziarie senza intermediari.
- **Real estate**: compravendita immobiliare automatizzata.
- **Gaming**: ownership di oggetti virtuali.

> Nota: Ethereum è la piattaforma principale per gli smart contract. Usa un linguaggio di programmazione (Solidity) specificamente progettato per scrivere contratti sicuri sulla blockchain.

CAPITOLO 8 – Model-Driven Engineering (MDE)

8.1 Cos'è un Modello?

Prima di capire il Model-Driven Engineering, è necessario capire cosa si intende per **modello**.

Un **modello** è una rappresentazione astratta e semplificata di un sistema, che cattura le caratteristiche rilevanti per un dato scopo, ignorando i dettagli irrilevanti. Non è mai completo: un modello che rappresentasse ogni aspetto del sistema *sarebbe* il sistema stesso.

Caratteristiche essenziali di un modello:

- **Mapping feature**: ogni modello è una rappresentazione di qualcosa (l'*original*).
- **Reduction feature**: il modello ignora attributi che non sono rilevanti per chi lo usa.
- **Pragmatic feature**: il modello è usato da persone o macchine al posto dell'originale, per uno scopo specifico.

Due tipi di modello:
- **Prescrittivo** (*prescriptive*): descrive come il sistema *deve essere* (blueprint per la costruzione).
- **Descrittivo** (*descriptive*): descrive come il sistema *è* (documentazione di qualcosa esistente).

Modello come first-class citizen

In ambienti moderni (MDE/MBSE) il modello è un **artefatto primario** di sviluppo, non semplice documentazione. Deve essere:
- Formalmente definito (con una sintassi e una semantica precisa).
- Processabile da macchine (generazione automatica di codice, analisi, test).
- Mantenuto aggiornato in sincronia con il sistema.

8.2 Model-Driven Engineering (MDE)

Definizione

Il **Model-Driven Engineering** è un approccio allo sviluppo software in cui i **modelli** sostituiscono (o integrano fortemente) il codice sorgente come principale artefatto di sviluppo. L'obiettivo è aumentare il livello di astrazione, rendere il processo più automatizzabile e migliorare la portabilità tra piattaforme.

Vantaggi principali:

- Riduzione della complessità accidentale (quella introdotta dagli strumenti e dalle piattaforme).
- Riuso dei modelli su piattaforme diverse tramite trasformazioni automatiche.
- Verifica formale del sistema prima di scrivere il codice.
- Generazione automatica di codice, test e documentazione dal modello.

MDE vs. MDA (Model-Driven Architecture)

Termine	Scopped By	Focus
MDE	Comunità di ricerca	Termine generale, include qualsiasi uso dei modelli come driver principale
MDA	OMG (Object Management Group)	Standard specifico dell'OMG che usa UML, MOF e QVT come fondamento tecnico

> **MDA** è uno *standard* che implementa la filosofia **MDE**. Il termine MDA è più comune nella pratica industriale.

8.3 Metamodello e Metapiramide

Gerarchia di modelli (M0–M3)

In MDE esiste una gerarchia di livelli di astrazione, chiamata **metapiramide** o stack M0–M3:

Livello	Nome	Contenuto	Esempio
M0	Istanza / Mondo reale	Oggetti concreti del sistema in esecuzione	L'oggetto `myOrder = { id: 42, prodotto: "libro" }`
M1	Modello	Struttura del sistema (classi, attributi)	Il diagramma UML con la classe `Order`
M2	Metamodello	Linguaggio per costruire modelli (cos'è una classe, un attributo...)	UML stesso: le regole che definiscono come `Class` e `Attribute` si relazionano
M3	Meta-metamodello	Linguaggio per costruire metamodelli	MOF (Meta-Object Facility): il linguaggio con cui è definito UML

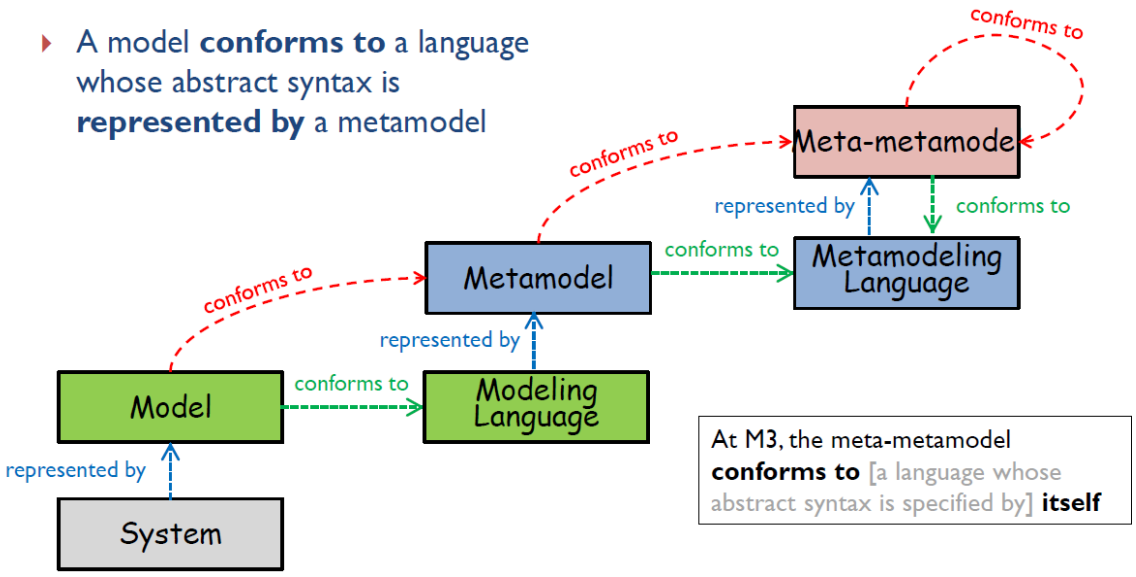
Chiave: ogni livello è un'istanza del livello superiore.

- `myOrder` (M0) è istanza di `Order` (M1).
- `Order` (M1) è istanza di `Class` in UML (M2).
- `Class` in UML (M2) è istanza di un metaconcetto in MOF (M3).

Metamodello = ontologia (insieme di concetti e di relazioni tra i concetti stessi)

Conformance

Si dice che un modello è **conforme** al suo metamodello se rispetta tutte le regole strutturali definite dal metamodello (“Un modello è conforme a un metamodello, come un linguaggio è conforme alla proprio grammatica”). La conformance è il concetto fondamentale per la validazione dei modelli.



8.4 Domain-Specific Language (DSL)

Cos'è un DSL?

Un **Domain-Specific Language** (DSL) è un linguaggio di modellazione (o programmazione) progettato per un dominio specifico, in contrapposizione a un General Purpose Language (GPL) come Java o Python.

Il DSL si divide in due componenti:

- **Sintassi astratta** (*abstract syntax*): definisce il vocabolario, i costrutti e le regole grammaticali del linguaggio. È la struttura logica del modello (es. il metamodello UML, definito tramite MOF).
- **Sintassi concreta** (*concrete syntax*): definisce la rappresentazione visibile del linguaggio:
  -*Concrete textual syntax* (es. il codice di un programma Java).
  -*Concrete graphical syntax* (es. i diagrammi UML).

Una stessa sintassi astratta può avere più sintassi concrete (es. UML può essere rappresentato sia graficamente che come XMI testuale).

GPL vs. DSL

Caratteristica	**GPL (es. Java, Python)**	**DSL (es. SQL, HTML, Simulink)**
Ambito	Generale	Specifico del dominio
Expressiveness	Alta	Limitata al dominio

Usabilità	Per esperti di programmazione	Accessibile agli esperti del dominio
Strumenti	Ampi	Specifici per il dominio

Esempi di DSL storici:

- **SQL**: per interrogare database relazionali.
- **HTML/CSS**: per strutturare e stilizzare pagine web.
- **MATLAB/Simulink**: per modellare sistemi di controllo.
- **Verilog/VHDL**: per descrivere circuiti digitali hardware.

8.5 OCL – Object Constraint Language

Motivazione

UML da solo non è abbastanza espressivo per specificare *tutti* i vincoli di un sistema. Ad esempio:

- "Il saldo di un conto non può essere negativo se il cliente non ha un fido."
- "Un dipendente non può essere il proprio responsabile."
- "L'età di un cliente deve essere maggiore di 18 anni."

Queste regole non possono essere espresse direttamente in un diagramma UML. Si usa quindi **OCL** (Object Constraint Language), un linguaggio formale preciso per esprimere vincoli e query sui modelli UML.

Tipi di espressioni OCL

Tipo	Scopo	Esempio
Invariant	Vincolo che deve essere sempre vero	`context Account inv: self.balance >= 0`
Precondition	Vincolo che deve essere vero *prima* dell'esecuzione di un'operazione	`context Account::withdraw(amount) pre: amount > 0`
Postcondition	Vincolo che deve essere vero *dopo* l'esecuzione	`post: self.balance = self.balance@pre - amount`
Query	Espressione che restituisce un valore dal modello	`context Account def: totalBalance(): Real = self.balance`

Caratteristiche di OCL

- **Side-effect free**: le espressioni OCL non modificano lo stato del modello.
- **Tipizzato**: ogni espressione ha un tipo (Boolean, Integer, String, Collection...).
- **Object-oriented**: si naviga il modello usando il punto (`.`) per accedere ad attributi e associazioni.
- **Basato su logica**: usa operatori booleani, quantificatori (`forAll`, `exists`), e collezioni (`select`, `collect`, `reject`).

Navigazione di collezioni in OCL

OCL gestisce collezioni in modo molto espressivo:

```
-- Tutti i dipendenti che guadagnano più di 50.000
context Company
def: highEarners(): Set(Employee) =
  self.employee->select(e | e.salary > 50000)

-- La somma di tutti gli stipendi
def: totalSalary(): Real =
  self.employee->collect(e | e.salary)->sum()

-- Verifica che tutti abbiano un manager
inv: self.employee->forAll(e | e.manager->notEmpty())
```

8.6 UML Profile

Motivazione

UML (Unified Modeling Language) è un linguaggio general-purpose per la modellazione. Ma in certi domini specifici (es. sistemi embedded, safety-critical, automotive), i costrutti standard di UML non bastano oppure sono troppo generici. Un **UML Profile** estende UML per adattarlo a un dominio specifico.

Un UML Profile usa tre meccanismi:

Meccanismo	Descrizione	Notazione
Stereotype	Aggiunge un nuovo "tipo" a un elemento UML esistente. Estende la classificazione.	`«nomeStereotype»`
Tagged Value	Aggiunge attributi aggiuntivi (proprietà) a un elemento stereotipato.	Attributi nel compartimento del tag

Constraint	Aggiunge regole (OCL o testo) che gli elementi devono rispettare.	`{regola}`

Esempio: SysML

SysML (*Systems Modeling Language*) è uno dei UML Profile più importanti. È pensato per il **Model-Based Systems Engineering (MBSE)** e aggiunge al UML diagrammi specifici per sistemi complessi:

Diagramma SysML	Descrizione
Block Definition Diagram (BDD)	Struttura del sistema: blocchi, proprietà, associazioni
Internal Block Diagram (IBD)	Connessioni interne tra le parti di un blocco
Requirement Diagram	Requisiti del sistema e le loro relazioni (non presente in UML standard)
Parametric Diagram	Vincoli matematici e fisici tra i parametri del sistema
Activity Diagram	Flussi di attività (analogo all'UML Activity Diagram)
Sequence Diagram	Interazioni temporali tra componenti
State Machine Diagram	Comportamento basato su stati
Use Case Diagram	Casi d'uso del sistema

Differenza chiave tra UML e SysML: UML nasce per il software (focus su classi e oggetti), SysML nasce per i sistemi (focus su blocchi, requisiti, flussi fisici).

8.7 MBSE – Model-Based Systems Engineering

Definizione

Il **MBSE** (Model-Based Systems Engineering) è l'applicazione formalizzata dei principi MDE all'ingegneria dei sistemi complessi (non solo software). È un'evoluzione rispetto al tradizionale *Document-Based SE*, in cui il design del sistema era descritto in documenti Word/PDF.

In MBSE:

- Il modello *è* la specifica, non la sua documentazione.
- Tutti gli stakeholder (ingegneri, manager, cliente) lavorano sullo stesso modello.
- Cambiamenti al modello si propagano automaticamente (no copia-incolla tra documenti).

- Il modello può essere usato per generare codice, test, simulazioni, documentazione.

Confronto Document-Based vs. Model-Based

Aspetto	Document-Based	Model-Based
Artefatto primario	Documenti Word/PDF	Modello formale (SysML, UML)
Consistenza	Difficile da mantenere	Garantita dallo strumento
Tracciabilità	Manuale	Automatica (link nel modello)
Analisi	Manuale	Automatica (simulazione, verifica)
Cambiamenti	Propagati manualmente	Propagati automaticamente

8.8 Trasformazioni di Modelli

Un aspetto fondamentale del MDE è la capacità di **trasformare** un modello in un altro. Le trasformazioni si classificano in:

Trasformazioni Model-to-Model (M2M)

Il modello sorgente viene trasformato in un altro modello. Ad esempio:
- Da un **PIM** (*Platform Independent Model*) a un **PSM** (*Platform Specific Model*): si specifica come il modello astratto viene adattato a una piattaforma concreta (es. Java, C++, ROS).
- Da un modello SysML a un modello AUTOSAR (per sistemi automotive).

Trasformazioni Model-to-Text (M2T)

Il modello viene trasformato in testo: codice sorgente, configurazioni, documentazione.

- **Forward engineering**: dal modello al codice (generazione automatica).
- **Reverse engineering**: dal codice al modello (ricostruzione del design).
- **Round-trip engineering**: si mantiene la sincronizzazione bidirezionale tra modello e codice.

Linguaggi di trasformazione

Standard	Tipo	Uso
QVT (Query/View/Transform)	M2M	Standard OMG, più diffuso nelle università
ATL (Atlas Transformation Language)	M2M	Più usato in pratica
Acceleo	M2T	Per la generazione di codice
Xtend / Xtext	M2T / DSL	Eclipse-based, molto usato in ambito industriale

---

8.9 Architettura CIM–PIM–PSM (MDA Standard)

Nel framework **MDA** ufficiale dell'OMG, si distinguono tre livelli di modello:

Livello	Acronimo	Descrizione
Computation Independent Model	CIM	Modello del dominio e dei requisiti, completamente indipendente da qualsiasi aspetto computazionale. Spesso espresso in linguaggio naturale o con diagrammi di casi d'uso.
Platform Independent Model	PIM	Modello della logica del sistema, indipendente dalla piattaforma di esecuzione. Tipicamente UML o SysML.
Platform Specific Model	PSM	Modello specifico per una piattaforma concreta (es. Java EE, .NET, ROS). Generato automaticamente dal PIM tramite trasformazione.

Flusso tipico:

```
Requisiti → CIM → (trasformazione) → PIM → (trasformazione) → PSM → Codice
```

In pratica, non sempre tutti e tre i livelli vengono usati. Spesso si parte direttamente dal PIM o si salta il CIM.

8.10 Strumenti MDE/MBSE più usati

Strumento	Tipo	Uso
Eclipse Modeling Framework (EMF)	Open source	Framework per costruire metamodelli e strumenti MDE in Eclipse
Papyrus	Open source	Editor UML/SysML basato su Eclipse
Enterprise Architect	Commerciale	UML, SysML, BPMN, molto usato in industria
Cameo Systems Modeler (MagicDraw)	Commerciale	SysML, MBSE, molto usato in aerospace e difesa
MATLAB/Simulink	Commerciale	Simulazione e generazione di codice per sistemi di controllo embedded
Rhapsody (IBM)	Commerciale	MDE per sistemi embedded e real-time

---

CAPITOLO 9 – Ingegneria dei Requisiti per i Sistemi Cyber-Fisici

9.1 Cos'è un Requisito?

Un **requisito** è una specifica per il sistema da costruire. È fondamentalmente un **contratto** tra il cliente e il team di sviluppo: descrive cosa il sistema deve fare (o non fare), stabilendo le aspettative in modo esplicito e verificabile.

> Un requisito mal scritto è uno dei motivi più comuni per cui i progetti software falliscono. Identificare gli errori nei requisiti *prima* di implementare il sistema è molto meno costoso che correggerli a sviluppo avviato.

Classificazione per destinatario

Tipo	Destinatario	Livello di dettaglio	Descrizione
User Requirements	Cliente, utenti finali	Alto (astratto)	Descrivono *cosa* il sistema deve fare in termini

			comprensibili ai non-tecnici
System Requirements	Sviluppatori	Basso (dettagliato)	Descrivono le funzioni, le caratteristiche e i vincoli del sistema in modo preciso

Classificazione per contenuto

Tipo	Descrizione	Esempio
Functional Requirements	Descrivono *cosa* il sistema deve fare	"Il sistema deve permettere all'utente di eseguire il login con email e password"
Non-Functional Requirements	Descrivono *come* il sistema deve farlo	"Il sistema deve rispondere alle richieste in meno di 200 ms nel 99% dei casi"

> **Attenzione**: i requisiti non-funzionali sono spesso quelli che generano più lavoro in fase di implementazione. Vincoli su sicurezza, performance, disponibilità e affidabilità rendono il codice molto più complesso. Devono essere **misurabili** e **non soggettivi**: "il sistema deve essere bello" non è un requisito valido.

9.2 Il Processo di Ingegneria dei Requisiti

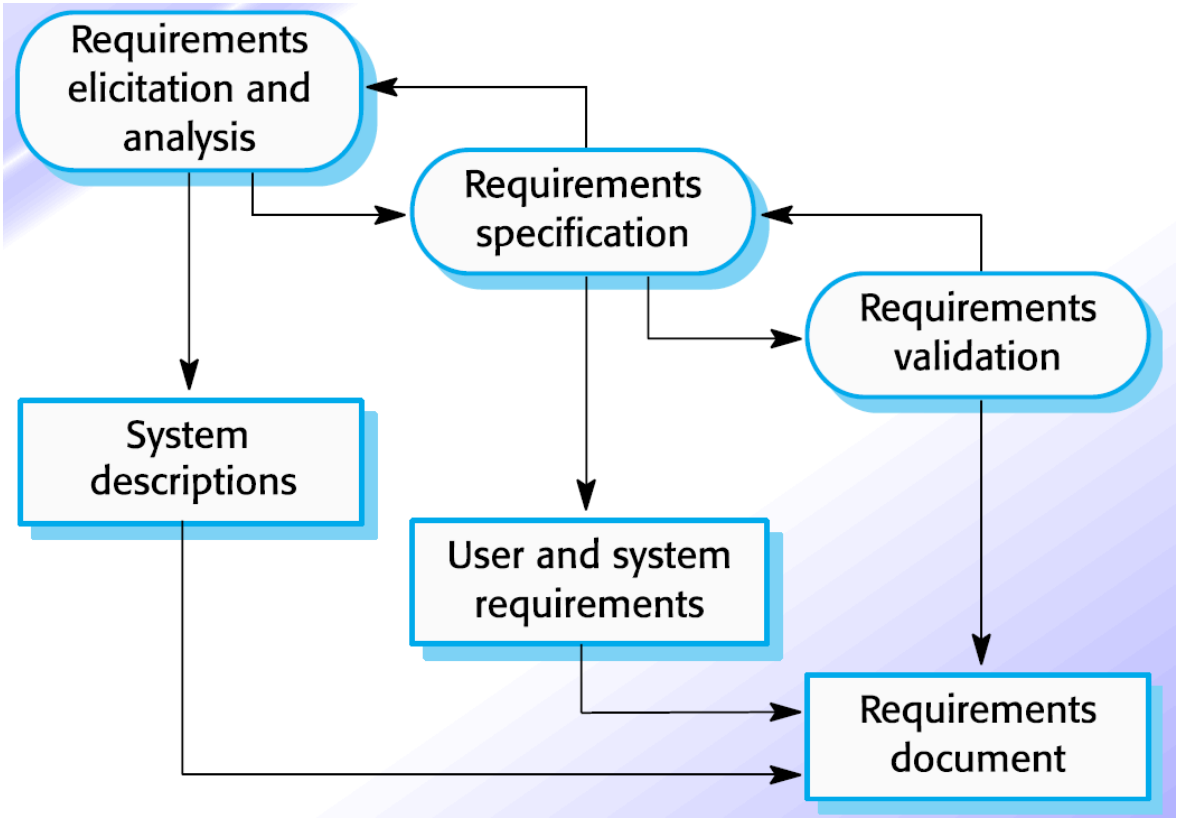
Il **Requirements Engineering (RE)** è il processo sistematico di definizione, documentazione e mantenimento dei requisiti nel ciclo di vita del sistema.

Le attività principali sono:

- **Elicitation** (raccolta): identificare i requisiti tramite interviste, osservazione, workshop, prototipazione.
- **Analysis** (analisi): verificare che i requisiti siano completi, consistenti, non ambigui e realizzabili.
- **Specification** (specifica): documentare i requisiti in modo formale o semi-formale.
- **Validation** (validazione): assicurarsi che i requisiti riflettano davvero le esigenze del cliente.

- **Management** (gestione): controllare i cambiamenti ai requisiti durante lo sviluppo.

> **Nota importante**: la raccolta dei requisiti non è banale. Non basta chiedere al cliente "cosa vuoi?". Spesso il cliente stesso non sa esattamente cosa vuole. Il Requirements Engineer deve saper fare le domande giuste, scoprire i bisogni impliciti, e distinguere tra bisogni reali e desideri velleitari.



9.3 Diagramma dei Casi d'Uso (Use Case Diagram)

Scopo e struttura

Il **diagramma dei casi d'uso** è uno degli strumenti più usati nell'elicitation dei requisiti. È parte di UML e descrive le interazioni tra gli attori esterni e il sistema.

Punti di forza:
- È comprensibile anche da persone non tecniche (clienti, stakeholder).
- Permette di validare rapidamente i requisiti con il cliente.
- È naturale da costruire in workshop con gli utenti.

Elementi

Elemento	Descrizione
Attore	Un'entità esterna al sistema che interagisce con esso. Può essere umana, un altro sistema o un dispositivo hardware. Rappresentata come un omino stilizzato.
Caso d'uso	Una funzionalità che il sistema offre a un attore. Rappresentato come un'ellisse con il nome.
Sistema	Il confine del sistema. Tutti i casi d'uso sono dentro il rettangolo del sistema; gli attori sono fuori.
Relazione di associazione	Linea che connette un attore a un caso d'uso che esso utilizza.
Relazione `<<include>>`	Un caso d'uso A include sempre il caso d'uso B come parte della sua esecuzione. Es: "Pagamento online" include sempre "Verifica carta".
Relazione `<<extend>>`	Un caso d'uso A *può* estendere B in certe condizioni. È A che decide se B viene attivato o meno. Es: "Invia email" può essere esteso da "Allega file".

Tipi di attori

- **Primario**: avvia l'interazione con il sistema (l'attore principale).
- **Secondario**: coinvolto nell'interazione ma non la avvia (es. un sistema esterno che viene consultato).
- **Attivo**: interagisce direttamente con il sistema.
- **Passivo**: viene coinvolto indirettamente.

Best practice

- **Non modellare il workflow**: il diagramma dice *cosa* fa il sistema, non *come* lo fa o in quale ordine. Usare diagrammi di sequenza o di attività per il workflow.
- **Gli attori devono essere esterni al sistema**: mai mettere un attore dentro il rettangolo del sistema.
- **Usare casi d'uso riassuntivi**: non andare nei dettagli implementativi all'interno del diagramma.
- **Nessuna decomposizione funzionale**: non usare il diagramma come se fosse un organigramma delle funzioni del sistema.
- **Se due attori accedono agli stessi casi d'uso**: crea un attore astratto se per quei casi d'uso é richiesto effettivamente solo 1 dei 2 attori(OR). Attenzione: nel caso in cui siano richiesti entrambi(AND) la connessione va fatta ai singoli attori, e non all' attore astratto.

9.4 Specifica dei Requisiti di Sistema (SRS)

La **System Requirements Specification (SRS)** è il documento (o modello) che contiene tutti i requisiti di sistema in modo formale. Esistono diversi stili di scrittura:

9.4.1 Linguaggio Naturale

Il modo più comune. Si usano parole chiave standard mutuate da RFC 2119:

Keyword	Significato
MUST	Requisito obbligatorio. Il sistema deve assolutamente farlo.
SHOULD	Requisito raccomandato ma non obbligatorio. È fortemente consigliato.
MAY	Requisito opzionale. Il sistema potrebbe farlo ma non è necessario.
MUST NOT	Il sistema non deve mai fare questa cosa.
SHOULD NOT	Il sistema non dovrebbe farlo, ma non è vietato.

> Nota: l'uso di **SHOULD NOT** e **MUST NOT** è sconsigliato dove possibile. È meglio formulare i requisiti in positivo (cosa si deve fare) piuttosto che in negativo (cosa non si deve fare).

Vantaggi: accessibile a tutti, flessibile.
Svantaggi: può essere ambiguo, difficile da verificare automaticamente.

9.4.2 Specifica Strutturata

Una variante del linguaggio naturale con una struttura più rigida: si usano template predefiniti con sezioni, moduli e campi obbligatori. Ogni requisito ha un formato standard (es. `ID | Descrizione | Priorità | Fonte | Criteri di accettazione`).

Vantaggi: meno ambiguità, più facile da gestire.
Svantaggi: richiede disciplina, meno flessibile.

9.4.3 Specifica Tabulare

Usa tabelle per descrivere le condizioni e le azioni corrispondenti. Particolarmente utile per descrivere comportamenti alternativi (es. "se la carta è scaduta, allora...").

Esempio:

Condizione	Azione
Carta valida, fondi sufficienti	Autorizza il pagamento
Carta valida, fondi insufficienti	Rifiuta e notifica l'utente
Carta scaduta	Rifiuta e chiedi di aggiornare i dati
Carta bloccata	Rifiuta e suggerisci di contattare la banca

9.4.4 Notazione Grafica (SysML)

SysML offre un **Requirement Diagram** dedicato, che permette di modellare i requisiti e le loro relazioni graficamente. Viene usato soprattutto in ambienti MBSE.

Relazioni tra requisiti in SysML:

- `«deriveReqt»`: un requisito deriva da un altro (più astratto).
- `«refine»`: un requisito raffina un altro (lo specifica con più dettaglio).
- `«satisfy»`: un blocco del modello soddisfa un requisito.
- `«verify»`: un test verifica che un requisito sia soddisfatto.
- `«trace»`: tracciabilità generica tra elementi.

Limite: il Requirement Diagram di SysML è spesso considerato uno strumento utile ma non sufficiente. Strumenti dedicati come **IBM DOORS** o **JAMA** sono preferiti in ambito industriale per la gestione dei requisiti.

9.5 Requisiti SMART

Un buon requisito deve essere **SMART**:

Lettera	Attributo	Descrizione
S	**Specific** (Specifico)	Non ambiguo. Deve descrivere un comportamento preciso, non una generalità.
M	**Measurable** (Misurabile)	Deve essere possibile verificare oggettivamente se il requisito è soddisfatto o no.
A	**Achievable** (Raggiungibile)	Deve essere tecnicamente possibile implementarlo con le risorse disponibili.

R	**Realistic** (Realistico)	Deve essere fattibile nel contesto specifico del progetto (budget, tempo, competenze).
T	**Traceable** (Tracciabile)	Deve essere possibile rintracciare ogni requisito fino alla sua fonte (stakeholder, documento, normativa).

Contromisura alla soggettività: i requisiti non-funzionali tendono ad essere soggettivi. La soluzione è aggiungere **criteri di accettazione** misurabili. Non "il sistema deve essere veloce" ma "il sistema deve rispondere entro 200ms al 99% delle richieste con un carico di 1000 utenti concorrenti".

9.6 Requisiti nei System of Systems

La gestione dei requisiti in un SoS è molto più complessa rispetto a un sistema monolitico, per diversi motivi:

Problemi specifici dei SoS

- **Scope incerto**: non è sempre chiaro dove finisce il SoS e dove inizia l'ambiente esterno. Ad esempio, i requisiti di un sistema ferroviario si estendono fino ai passeggeri? Ai servizi di emergenza? Alle autorità di regolazione?

- **Requisiti emergenti**: alcune proprietà del SoS emergono solo quando i CS interagiscono tra loro e non possono essere derivate dai requisiti dei singoli CS.

- **Evoluzione non coordinata**: ogni CS può evolvere in modo indipendente, rendendo difficile mantenere i requisiti del SoS allineati con quelli dei singoli CS.

- **Proprietari multipli**: ogni CS può avere un proprietario diverso con obiettivi e priorità diverse. Non esiste un'unica autorità decisionale.

Approcci per la gestione dei requisiti in SoS

- **Goal-Based Requirements**: si specificano i requisiti in termini di *goal* (obiettivi) piuttosto che di funzionalità precise. I CS sono liberi di scegliere come raggiungere il goal. Questo approccio è più flessibile e si adatta meglio all'evoluzione.

- **Scenario-Based Requirements**: si usano scenari concreti (casi d'uso, racconti utente) per elicitare e validare i requisiti. Gli scenari coinvolgono tutti gli stakeholder e rendono visibili le interazioni tra CS.

- **Traceability continua**: si mantiene una mappa di tracciabilità che collega ogni requisito del SoS ai requisiti dei singoli CS che lo soddisfano. Questa mappa deve essere aggiornata a ogni cambiamento.

9.7 Verifica e Validazione dei Requisiti

La **verifica** e la **validazione** si applicano anche ai requisiti stessi, non solo al sistema:

- **Verifica dei requisiti**: i requisiti sono internamente consistenti? Non ci sono contraddizioni? Sono completi? Ogni requisito è verificabile?

- **Validazione dei requisiti**: i requisiti riflettono davvero le esigenze degli stakeholder? Se implementassimo esattamente questi requisiti, il cliente sarebbe soddisfatto?

Tecniche di verifica e validazione

Tecnica	Descrizione
Inspection / Review	Un gruppo di esperti legge i requisiti e identifica problemi, ambiguità, omissioni
Prototipazione	Si costruisce un prototipo per far vedere al cliente qualcosa di concreto, così da validare le aspettative
Test-driven requirements	Ogni requisito è accompagnato da un test di accettazione che deve passare per dichiarare il requisito soddisfatto
Simulazione	Per sistemi CPS, si può simulare il comportamento del sistema con i requisiti proposti prima di costruirlo
Formal verification	Si usa la matematica formale (es. model checking) per verificare proprietà sui requisiti

---

9.8 Interoperabilità nei Modelli

Un concetto critico in MDE applicato ai CPS è l'**interoperabilità** tra modelli. IEEE la definisce come:

> La capacità di due o più sistemi o componenti di scambiare informazioni e di usare le informazioni scambiate.

In Model-Driven Engineering, l'interoperabilità è fondamentale perché si usano tool e linguaggi diversi per aspetti diversi del sistema (es. SysML per la struttura, Modelica per la simulazione fisica, AADL per il real-time). Per poter trasformare un modello in un altro è necessario che entrambi i modelli siano conformi a metamodelli che condividono almeno una base comune (es. MOF).

Livelli di interoperabilità

Livello	Descrizione
Sintattica	I modelli usano lo stesso formato fisico (es. XML/XMI)
Strutturale	I modelli condividono elementi strutturali (es. stesso metamodello)
Semantica	I modelli concordano sul significato degli elementi (più difficile da garantire)
Pragmatica	I modelli possono essere usati insieme per uno scopo comune

9.9 Caso Studio Pratico: Sistema ALARP

Per capire meglio come si scrivono i requisiti usando le keyword formali (MUST, SHOULD) e il concetto di SIL (Safety Integrity Level), analizziamo un sistema fittizio discusso a lezione: il **sistema ALARP** (Advanced Local Alerting for Railway Personnel).

Contesto: Un sistema di allarme deve avvisare gli operai ferroviari dell'arrivo di un treno. Esistono due zone:

- **Red Zone**: zona ad alto rischio (sui binari).
- **Green Zone**: zona a basso rischio (vicino ai binari ma a distanza di sicurezza).

Esempi di Requisiti di Sistema (SRS) estratti:

- `[REQ 01 M]` Il sistema ALARP **MUST** essere in grado di determinare la posizione esatta dei lavoratori rispetto ai binari.
- `[REQ 03 M]` Nel caso di lavoratori in Red Zone, il sistema ALARP **MUST** attivare un allarme acustico e visivo con un preavviso di almeno 30 secondi prima dell'arrivo del treno.
- `[REQ 04 M]` Nel caso di lavoratori in Green Zone, il sistema ALARP **SHOULD** avvisare i lavoratori del transito, ma senza richiedere un allarme acustico obbligatorio.

- `[REQ 17 M]` In Red Zone, l'avviso di allarme del sistema ALARP **MUST NOT** poter essere silenziato manualmente dagli operai finché il treno non è transitato.
- `[REQ 19 M]` Il sistema ALARP **MUST** avere un SIL 2 secondo la normativa EN 50128 per la gestione degli allarmi in Red Zone.

> **Analisi didattica**: Notare come la parola `MUST` sia usata per vincoli di sicurezza assoluti (allarme in zona rossa, impossibilità di spegnerlo), mentre `SHOULD` viene usato per comportamenti raccomandati ma non critici (avviso in zona verde). Il requisito 19 è un **requisito non-funzionale** che vincola l'affidabilità del software a uno standard esterno.

CAPITOLO 10 – Ciclo di Vita del Software (System Lifecycle)

10.1 Introduzione ai Modelli di Processo

Un **modello di processo software** è una rappresentazione astratta di come il software viene sviluppato: quali attività vengono svolte, in quale ordine e come si relazionano tra loro. Non esiste un modello universale: la scelta dipende dal tipo di sistema, dai requisiti di sicurezza e dalla maturità del team.

I tre modelli principali trattati nel corso sono:

Modello	Caratteristica principale
Waterfall	Plan-driven. Fasi separate e distinte: prima si specifica, poi si sviluppa.
Incremental Development	Specifica, sviluppo e validazione sono interlacciati. Piano adattivo.
Integration and Configuration	Il sistema viene assemblato da componenti esistenti e configurabili (COTS).

> **Nota importante**: nella pratica, i sistemi grandi usano sempre una combinazione di questi modelli. "Agile" non significa "meno strutturato" – anche se purtroppo in pratica spesso viene inteso così.

10.2 Il Modello Waterfall

Struttura

Il **modello waterfall** (cascata) è il modello plan-driven per eccellenza. Le sue fasi si susseguono in sequenza e ogni fase deve essere completata prima di passare alla successiva:

- **Requirements analysis and definition**: raccolta e analisi dei requisiti, produzione del documento di specifica.
- **System and software design**: progettazione dell'architettura e dei componenti.
- **Implementation and unit testing**: scrittura del codice e test dei singoli componenti.
- **Integration and system testing**: integrazione di tutti i componenti e test del sistema completo.
- **Operation and maintenance**: rilascio, operatività e correzione dei difetti.

Contesti di utilizzo

Il waterfall è particolarmente adatto quando:

- Il sistema è sviluppato in più sedi geografiche (il piano rigido facilita il coordinamento).
- I requisiti sono ben compresi e non cambieranno significativamente durante lo sviluppo.
- **La sicurezza è un requisito critico** (safety-critical systems: ferroviario, aerospaziale, medicale).

Problemi del Waterfall

Il difetto principale del waterfall è la **difficoltà di gestire i cambiamenti** una volta che il processo è avviato:

- In linea di principio, non si può tornare a una fase precedente senza invalidare tutto il lavoro successivo.
- La partizione rigida in fasi distinte rende difficile rispondere a requisiti che cambiano in corso d'opera.
- Non è adatto quando i requisiti non sono chiari fin dall'inizio.

10.3 Sviluppo Incrementale (Incremental Development)

Struttura

Nello **sviluppo incrementale**, le attività di specifica, sviluppo e validazione sono *interlacciate*: anziché fare tutto in sequenza, si lavora per incrementi. Ogni incremento

aggiunge un sottoinsieme di funzionalità al sistema, ottenendo versioni parziali ma funzionanti.

Vantaggi

- **Costo ridotto per gestire i cambiamenti**: il rework in caso di cambio di requisiti è molto inferiore rispetto al waterfall – si riscrive un incremento, non tutto il progetto.
- **Feedback più facile dal cliente**: il cliente può vedere e usare versioni intermedie del sistema e dare feedback concreto.
- **Prioritizzazione delle funzioni più importanti**: le funzionalità più critiche vengono sviluppate nei primi incrementi e sono disponibili prima.

Problemi

- **Il processo non è visibile**: senza deliverable documentati a ogni step, i manager faticano a misurare il progresso. Produrre documentazione per ogni versione è costoso.
- **La struttura del sistema degrada nel tempo**: aggiungere incrementi senza dedicare tempo al refactoring tende a corrompere l'architettura, rendendo i futuri cambiamenti sempre più costosi.

10.4 Integrazione e Configurazione (Reuse-Oriented)

Struttura

Questo approccio si basa sul **riuso del software**: il sistema viene assemblato da componenti esistenti e configurabili, invece di essere sviluppato da zero.

Componenti riusabili tipici:

- **COTS** (Commercial Off-The-Shelf): sistemi commerciali interi acquistati e integrati.
- **Librerie e pacchetti SW**: es. librerie J2EE, librerie Python, framework.
- **Web services**: servizi esposti tramite API standard su internet.

Vantaggi e Svantaggi

Aspetto	Commento
✅ **Costi ridotti**	Meno codice da scrivere da zero = meno lavoro, meno difetti da introdurre
✅ **Delivery più rapida**	Componenti già testati e documentati
❌ **Compromessi sui requisiti**	Il sistema potrebbe non soddisfare esattamente le esigenze reali se i componenti non si adattano perfettamente

❌ **Perdita di controllo**	Non si controlla l'evoluzione dei componenti di terze parti
❌ **Rischi nascosti**	Funzionalità non documentate, vulnerabilità di sicurezza, performance non garantite nei componenti COTS

10.5 Le Quattro Attività Fondamentali

Indipendentemente dal modello di processo scelto, ogni sviluppo software include quattro macro-attività:

Attività	Descrizione	Posizione nel processo
Specification	Definire cosa deve fare il sistema (Requirements Engineering)	Prima nel waterfall; interlacciata nell'incrementale
Development	Progettare e implementare il sistema	Dopo i requisiti nel waterfall; interlacciata nell'incrementale
Validation	Verificare che il sistema soddisfi i requisiti (V&V)	Ultima nel waterfall; interlacciata nell'incrementale
Evolution	Modificare il sistema al cambiare dei requisiti	Post-rilascio nel waterfall; continua nell'incrementale

Il processo di specifica (Requirements Engineering)

Il processo di requirements engineering è esso stesso articolato in sotto-attività:

- **Feasibility study**: è tecnicamente e finanziariamente fattibile costruire il sistema?
- **Requirements elicitation and analysis**: cosa fanno gli stakeholder? Cosa vogliono?
- **Requirements specification**: documentazione dei requisiti user e system.
- **Requirements validation**: i requisiti definiscono davvero il sistema che il cliente vuole?

Design e Implementazione

Le attività di design includono:

- **Architectural design**: struttura globale del sistema, componenti principali e loro relazioni.
- **Database design**: struttura dei dati e loro rappresentazione.
- **Interface design**: definizione delle interfacce tra componenti.

- **Component selection and design**: ricerca di componenti riusabili; se non disponibili, progettazione da zero.

10.6 Verifica e Validazione (V&V)

Definizioni precise (PMBOK)

- **Validation**: assicurare che il prodotto soddisfi le *esigenze* del cliente e degli stakeholder. Tipicamente coinvolge il cliente. → *Stiamo costruendo il sistema giusto?*
- **Verification**: verificare che il prodotto sia conforme a una regolamentazione, specifica o requisito. Tipicamente è un processo interno. → *Stiamo costruendo il sistema nel modo giusto?*

Fasi di test

Il testing si articola in tre fasi:

Fase	Oggetto del test
Component testing (Unit testing)	Singoli componenti o oggetti testati in isolamento
System testing (Integration testing)	Il sistema nel suo insieme. Particolarmente importante per le proprietà emergenti.
Customer testing (Acceptance testing)	Il sistema viene testato con dati reali del cliente per verificare che soddisfi le sue esigenze

---

10.7 Il Modello V (V-Model)

Il **V-Model** è una variante plan-driven che mostra esplicitamente la corrispondenza tra le attività di sviluppo (lato sinistro della V) e le attività di test (lato destro della V):

```
Requisiti di sistema ←————————————→ Acceptance testing
  Requisiti SW   ←——————————————→ System testing
    Architettura    ←————————————→ Integration testing
      Moduli        ←————————→ Unit testing
                        Codice
```

Ogni attività di sviluppo produce un artefatto che verrà poi verificato da una corrispondente attività di test. Questo garantisce la **tracciabilità** tra requisiti e test.

10.8 Case Study: EN 50128/50129 (Sistemi Ferroviari)

Il corso fa riferimento alla norma **EN 50128** come caso studio di processo industriale reale.

EN 50128 è la norma europea per il software dei sistemi di segnalazione ferroviaria. Definisce un processo di sviluppo specifico per software usato in sistemi di controllo e protezione ferroviari.

Caratteristiche principali della norma:
- Prende il **waterfall come modello di riferimento** (con possibilità di iterazioni limitate).
- Distingue **SIL** (Safety Integrity Level) da 0 a 4: più alto il SIL, più rigoroso il processo richiesto.
- Richiede documentazione dettagliata per ogni fase: documenti di specifica, progettazione, test, validazione.
- Le attività di V&V sono obbligatorie e indipendenti dallo sviluppo.

EN 50129 è la norma equivalente a livello di sistema (non solo software).

Entrambe le norme seguono il **V-cycle** come schema di riferimento, con le fasi di sviluppo a sinistra e le fasi di V&V a destra.

> **Nota**: EN 50128 è parte della famiglia di norme IEC 61508 (la norma madre per la functional safety nei sistemi controllati da elettronica). La famiglia IEC 61508 è il riferimento plan-driven per eccellenza in ambito safety-critical.

CAPITOLO 11 – Risk Assessment e Hazard Analysis

11.1 Gestione del Rischio

Definizione

La **gestione del rischio** (Risk Management) si occupa di identificare i rischi e definire piani per minimizzare il loro effetto su un progetto o un sistema.

Nei CPS è particolarmente critica perché l'integrazione tra componente cyber e componente fisica crea rischi che non esistono nei sistemi puramente software.

Fonti di incertezza (da cui nascono i rischi):

- Requisiti poco chiari o che cambiano durante lo sviluppo.
- Difficoltà di stimare tempi e risorse.
- Differenze nelle competenze individuali del team.
- Dipendenze da componenti di terze parti.

Strategia migliore: anticipare i rischi, analizzarne l'impatto e adottare misure preventive *prima* che si materializzino.

11.2 Classificazione dei Rischi

I rischi si classificano lungo due dimensioni:

Per oggetto dell'impatto

Tipo	Cosa subisce l'impatto	Esempio
Project risk	Tempi e risorse del progetto	Turn-over del personale, ritardi hardware
Product risk	Qualità o performance del sistema	Bug non rilevati, mancato soddisfacimento dei requisiti non-funzionali → **è questo il nostro focus!**
Business risk	L'organizzazione stessa	Cambio tecnologico che rende obsoleto il prodotto prima del rilascio

Per tipo (root cause)

Tipo	Esempi di rischio
Technology	Il DB non gestisce il numero di transazioni previsto
Organizational	L'azienda si riorganizza cambiando le priorità del progetto
People	Impossibile assumere personale con le competenze necessarie
Requirements	Richieste di modifica che comportano una riprogettazione dell'architettura
Estimation	Il tempo/costo di sviluppo è stato sottostimato
Tool	Il code generator produce codice inefficiente

11.3 Il Processo di Risk Management

Il processo si articola in quattro fasi principali:

Fase 1 – Risk Identification

Identificare tutti i rischi rilevanti. Si usano:

- Checklist standard (basate su esperienza storica).
- Workshop del team.
- Processi organizzativi standardizzati (es. CMMI).

Fase 2 – Risk Analysis

Per ogni rischio identificato, stimare:

- **Probabilità** di accadimento: Very Low / Low / Moderate / High / Very High.
- **Conseguenze** (severità): Catastrophic / Serious / Tolerable / Insignificant.

Il prodotto (concettuale) di probabilità e severità dà il **livello di rischio**.

Fase 3 – Risk Planning

Definire una strategia per ogni rischio rilevante:

Strategia	Obiettivo	Esempio
Avoidance	Ridurre la probabilità che il rischio si realizzi	Usare tecnologie con track record provate
Minimization	Ridurre l'impatto del rischio sul progetto	Avere un piano di backup per il personale chiave
Contingency	Prepararsi a gestire il rischio se si materializza	Piano di emergenza se un fornitore chiude

Fase 4 – Risk Monitoring

Rivalutare periodicamente ogni rischio identificato:

- La probabilità è aumentata o diminuita?
- Le conseguenze sono cambiate?
- I rischi vengono discussi regolarmente nelle riunioni di progetto.
- Il monitoraggio è necessario perché il sistema, l'ambiente e l'operatività evolvono nel tempo.

11.4 Risk Assessment nel Ciclo di Vita

Quando fare il Risk Assessment?

Il Risk Assessment si posiziona naturalmente all'interno di un processo **plan-driven** (tipicamente nelle fasi iniziali, prima che il design sia completato).

Perché? Perché il Risk Assessment è strettamente legato all'analisi della completezza dei **requisiti non-funzionali** di dependability e sicurezza:

- Come risultato del RA, nuovi requisiti non-funzionali vengono associati alle funzioni del sistema.
- Aggiungere requisiti di disponibilità stretta *a posteriori* (es. a metà sviluppo) può richiedere una riprogettazione completa dell'architettura. Meglio farlo prima!

Nella norma EN 50126 (famiglia sicurezza ferroviaria), il Risk Assessment avviene principalmente nelle fasi di Concept e System Definition (fasi 3 e 4 del ciclo di vita della norma).

11.5 Hazard Analysis and Risk Assessment (HARA)

Differenza tra Risk Assessment e Hazard Analysis

La **Hazard Analysis (HA)** è una tecnica specifica per sistemi fisici: identifica i pericoli (*hazard*) potenziali del sistema. L'analisi degli hazard si combina con la valutazione del rischio per ottenere la **HARA** (Hazard and Risk Assessment).

> La HA si applica sia ai sistemi in funzione che durante la progettazione. **Non è solo per la safety**: può riguardare operatività, interfacce, manutenibilità.

Perché fare HARA?

- In molti settori è un **obbligo di legge** (es. sistemi ferroviari in Europa).
- Per ridurre il rischio complessivo del sistema nel lungo periodo.
- Per tenere sotto controllo elementi che potrebbero introdurre nuovi rischi in futuro.

11.6 HAZOP – Hazard and Operability Study

Cos'è

Il **HAZOP** (Hazard and Operability Study, o semplicemente HA nel corso) è la tecnica più usata per l'analisi degli hazard nei sistemi industriali.

- La norma di riferimento è **IEC 61882** (Hazard and Operability Studies – Application Guide).
- È un approccio **system-centered**: analizza il sistema nel suo insieme, non i singoli componenti (a differenza della FMEA che è component-centered).
- Identifica possibili **deviazioni dall'intento di progetto** (*design intent*) e per ciascuna:
- Trova le potenziali *cause* della deviazione.
- Deduce le possibili *conseguenze*.

> **Il "segreto" del HAZOP**: si usano **guide words** per stimolare la ricerca creativa di tutte le possibili deviazioni. Si considera sistematicamente ogni possibile evento negativo, poi si definiscono i rischi associati e le mitigazioni.

11.7 Cos'è un Hazard

Definizione (ISO IEC 61882)

Un **hazard** è una *"potenziale fonte di danno"*.

- **Causa dell'hazard**: una condizione o attività che, se non controllata, può causare conseguenze indesiderate.
- **Conseguenza dell'hazard**: danno fisico a persone, danni ambientali, perdita di equipaggiamento o proprietà.

Definizione alternativa: *"Una condizione reale o potenziale che, quando attivata, può trasformarsi in una serie di eventi interrelati che risultano in danni a equipment o persone."*

Origini degli hazard (M.M.M.M.E. – 5M)

Fonte	Categoria	Esempi
Man	Fattore umano	Azioni non sicure, errori operativi
Machinery	Macchine	Layout, design dell'equipment
Materials	Materiali	Sostanze chimiche, gas
Method	Metodo	Il modo in cui le persone svolgono il lavoro

Environment	Ambiente	Qualità dell'aria, rumore, vibrazione, ventilazione

Quando vengono identificati gli hazard?

Gli hazard vengono identificati tipicamente nelle fasi:
- **Concept**: analisi preliminare del sistema e del suo contesto operativo.
- **System definition**: definizione delle condizioni d'applicazione e dei confini.
- **Risk analysis**: analisi formale dei rischi.

> L'intera fase di gestione degli hazard si estende però a tutto il lifecycle del sistema.

11.8 Il Processo HARA Passo per Passo

Il processo HARA si articola in **tre step**:

Step 1 – Hazard Identification

Prima parte: definizione chiara del sistema da analizzare:
- Dominio del sistema.
- Interfacce del sistema con l'ambiente esterno.
- Funzionalità del sistema.
- Operatività (installazione, configurazione, manutenzione).

Seconda parte: analisi della definizione del sistema per ogni elemento definito, utilizzando le **guide words** del HAZOP per identificare le possibili deviazioni.

Come si svolge: tipicamente come **workshop** (sessione di lavoro con un team multidisciplinare). I partecipanti hanno background diversi (ingegneria, safety, operazioni, cliente). Il risultato è una raccolta di fogli che descrivono il sistema e gli hazard identificati.

Step 2 – Hazard Analysis – Identificazione delle Mitigazioni

Una volta trovati gli hazard, si definiscono le **mitigazioni**:

- Le mitigazioni puntano a **ridurre la severità** delle conseguenze o **ridurre la frequenza** di accadimento.
- Possono essere **tecniche** (es. modifica del design del sistema, aggiunta di ridondanza) o **procedurali** (es. richiedere all'utente di seguire una procedura operativa definita).

Domanda chiave: deve essere trovata una mitigazione per ogni hazard? E quanta mitigazione è "sufficiente"? → La risposta dipende dal livello di rischio calcolato nel passo successivo.

Step 3 – Risk Assessment

Per ogni hazard (o gruppo di hazard correlati, detti *macro-hazard*), si valuta il rischio combinando:

- **Frequenza** (likelihood) di accadimento dell'hazard.
- **Severità** delle conseguenze.

Il rischio viene derivato tramite una **Risk Matrix**.

11.9 Scala di Frequenza (Likelihood)

Livello	Definizione
Frequent	È probabile che l'hazard avvenga frequentemente (continuamente)
Probable	Avverrà più volte nel ciclo di vita del sistema
Occasional	È probabile che avvenga molte volte nel ciclo di vita
Remote	È ragionevole aspettarsi che avvenga qualche volta nel ciclo di vita
Improbable	Improbabile, ma possibile. Si può supporre che avvenga eccezionalmente
Incredible	Estremamente improbabile. Si può supporre che non avvenga

---

11.10 Scala di Severità

Orientata alla safety (focus su persone e ambiente)

Categoria	Severità delle conseguenze
Insignificant	Possibili lesioni minori
Marginal	Più lesioni minori e/o minaccia significativa all'ambiente

Critical	Singola vittima e/o gravi lesioni e/o danno significativo all'ambiente
Catastrophic	Vittime multiple e/o gravi lesioni multiple e/o ingente danno ambientale

> Esistono anche scale di severità orientate alla **RAM** (Reliability, Availability, Maintainability) e alla **finanza**, per domini in cui il danno principale non è alla salute umana ma alla continuità operativa o al business.

11.11 La Risk Matrix

La **Risk Matrix** combina frequenza e severità per derivare il livello di rischio:

Livello di rischio	Azione richiesta
Intolerable	Deve essere eliminato (nessuna accettazione possibile)
Undesirable	Accettabile solo se la riduzione del rischio è impraticabile, e solo con accordo esplicito dell'autorità competente
Tolerable	Accettabile con controlli adeguati e accordo dell'autorità
Negligible	Accettabile senza necessità di accordi formali

> **Nota**: non esiste una risk matrix universale. La matrice (con le sue scale di frequenza, severità e criteri di accettabilità) deve essere **progettata ad-hoc** per il progetto specifico. Diversi domini (ferroviario, medico, aerospaziale) hanno criteri diversi.

11.12 Valutazione Qualitativa vs. Quantitativa

Valutazione Qualitativa (simile a FMECA)

- Si basa principalmente sul **giudizio degli esperti di dominio** e sull'esperienza passata.
- Le stime sono tipicamente agli ordini di grandezza.
- **Vantaggi**: semplice, meno costosa, non richiede raccolta dati dettagliata.
- **Svantaggi**: le assunzioni devono essere documentate; trovare argomenti solidi per la likelihood può essere difficile (si tende ad essere conservativi).

Valutazione Quantitativa (con Fault Tree)

- Usa modelli formali con dati obiettivi e validati.
- **Vantaggi**: più accurata, migliore comprensione di cause e conseguenze.
- **Svantaggi**: complessa, richiede esperti, richiede molti dati, difficile quantificare la probabilità di *systematic failures* (vs. *random failures*).

> **Uso dei dati storici**: il RA si basa sempre su qualche forma di estrapolazione dal passato verso il futuro. I dati storici devono essere usati con cautela: le circostanze del passato potrebbero non essere rappresentative di quelle future, specialmente per eventi rari (incidenti gravi).

11.13 Hazard Log

Il **Hazard Log** è il documento (o database) centrale che raccoglie tutti i risultati della HARA e ne mantiene lo stato nel tempo.

Contenuto del Hazard Log

Campo	Descrizione
ID	Codice identificativo dell'hazard
Date	Data di identificazione
Hazard	Descrizione testuale dell'hazard
Cause	Scenario di generazione dell'hazard
Consequence	L'incidente che potrebbe derivare dall'hazard
Mitigation	Descrizione di come l'hazard è stato mitigato
Resp.	Persona/entità responsabile della mitigazione
Action	Operazioni aggiuntive da svolgere
Ref.	Riferimenti ai documenti di dettaglio
Status	Stato corrente dell'hazard

Stato degli Hazard

Status	Definizione
Open	Le azioni per chiudere l'hazard non sono ancora state formalmente concordate

Closed	Le azioni sono state completate e documentate (inserite nei requisiti)
Solved	Le azioni sono state concordate ma non ancora completate/documentate
Cancelled	Non è considerato un hazard, oppure è contenuto in un altro hazard già identificato
Transferred	L'hazard non rientra nei limiti del sistema in esame (è responsabilità di un altro sistema)

> Il Hazard Log è la base per la **gestione continuativa del rischio**: viene aggiornato ad ogni cambiamento significativo del sistema durante il ciclo di vita, non solo in fase di progettazione.

11.14 Guide Words HAZOP

Le **guide words** del HAZOP sono parole chiave standardizzate che stimolano la ricerca creativa delle deviazioni. Quando applicate alle *funzioni* del sistema:

Guide Word	Significato applicato alle funzioni
NOT	La funzione non viene eseguita (es.: il controllo d'integrità dei dati non viene eseguito)
MORE	Aumento quantitativo della funzione/dato (es.: viene allocata più memoria del previsto)
LESS	Diminuzione quantitativa (es.: una funzione viene eseguita meno frequentemente del dovuto, un valore è più piccolo del previsto)
PART OF	La funzione viene eseguita solo in parte (es.: output parziale o esecuzione parziale)
OTHER THAN	La funzione si esegue ma produce un risultato diverso da quello atteso
LATE	La funzione avviene in ritardo rispetto al tempo previsto
EARLY	La funzione avviene in anticipo rispetto al tempo previsto

Domande da porsi per ogni funzione

Per ogni funzione identificata nel sistema, il team HAZOP risponde alle seguenti domande:

- **Cosa può andare storto?** (quale hazard può avvenire) → basato sulle guide words.
- **Come potrebbe accadere?** (causa dell'hazard) → basato sulla conoscenza del dominio.
- **Quali sono le conseguenze?** (severità) → analisi dell'impatto.
- **Con che probabilità avviene?** (likelihood) → spesso la domanda più difficile.

APPENDICE – Acronimi e Glossario

Acronimo	Significato
ABS	Anti-lock Braking System
ACK	Acknowledgement
AADL	Architecture Analysis and Design Language
AP	Availability + Partition Tolerance (scelta CAP)
ATL	Atlas Transformation Language
BFT	Byzantine Fault Tolerance
BDD	Block Definition Diagram (SysML)
CA	Certificate Authority
CAP	Consistency, Availability, Partition Tolerance
CIM	Computation Independent Model
CP	Consistency + Partition Tolerance (scelta CAP)
CPS	Cyber-Physical System
CS	Constituent System
DHT	Distributed Hash Table
DSL	Domain-Specific Language
ECU	Electronic Control Unit
EMF	Eclipse Modeling Framework
FD	Failure Detector
FIFO	First In, First Out
FLP	Fischer, Lynch, Paterson (teorema)
GPS	Global Positioning System
IaaS	Infrastructure as a Service
IBD	Internal Block Diagram (SysML)
M2M	Model-to-Model (trasformazione)
M2T	Model-to-Text (trasformazione)
MDA	Model-Driven Architecture
MDE	Model-Driven Engineering
MBSE	Model-Based Systems Engineering
MOF	Meta-Object Facility
NFT	Non-Fungible Token
NTP	Network Time Protocol
OCL	Object Constraint Language
OMG	Object Management Group
PaaS	Platform as a Service

PAR	Positive Acknowledgement with Retransmission
PBFT	Practical Byzantine Fault Tolerance
PIM	Platform Independent Model
PoET	Proof of Elapsed Time
PoS	Proof of Stake
PoW	Proof of Work
PSM	Platform Specific Model
QVT	Query/View/Transform
RE	Requirements Engineering
RUI	Relied Upon Interface
RUMI	Relied Upon Message Interface
RUPI	Relied Upon Physical Interface
RUHI	Relied Upon Human Interface
RUS	Relied Upon Service
SaaS	Software as a Service
SLA	Service Level Agreement
SoA	Service Oriented Architecture
SoS	System of Systems
SRS	System Requirements Specification
SysML	Systems Modeling Language
TAI	International Atomic Time
TDMA	Time Division Multiple Access
TEE	Trusted Execution Environment
TOF	Time of Flight
TSI	Time Synchronization Interface
TT	Time-Triggered
UTC	Universal Time Coordinated
WCET	Worst Case Execution Time
2PC	Two-Phase Commit